

Unified Modeling Language (UML)

Prof. Dr. Lucas C. Ribas

Disciplina: Programação Orientada a Objetos

Departamento de Ciências de Computação e Estatística



UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"



IBILCE / UNESP - CÂMPUS DE SÃO JOSÉ DO RIO PRETO



- ◎ Histórico da UML
- ◎ Diagrama de classes
- ◎ Representação de classes
 - Atributos e métodos
 - Tipos de acesso e modificadores
- ◎ Relacionamentos entre classes
 - Herança, Implementação, Associação, Agregação e Composição

2



- ◎ **UML** (Linguagem de Modelagem Unificada) é uma linguagem visual
 - **Análise e projeto** de sistemas computacionais no paradigma de **Orientação a Objetos**
- ◎ Nos últimos anos, a UML se tornou a linguagem padrão de projeto de software, adotada internacionalmente pela indústria de Engenharia de Software



- UML não é uma linguagem de programação
- É uma linguagem de modelagem, utilizada para representar o sistema de software sob os seguintes aspectos:
 - Requisitos
 - Comportamento
 - Estrutura lógica
 - Dinâmica de processos
 - Comunicação/Interface com os usuários



◎ Por que modelar um sistema?

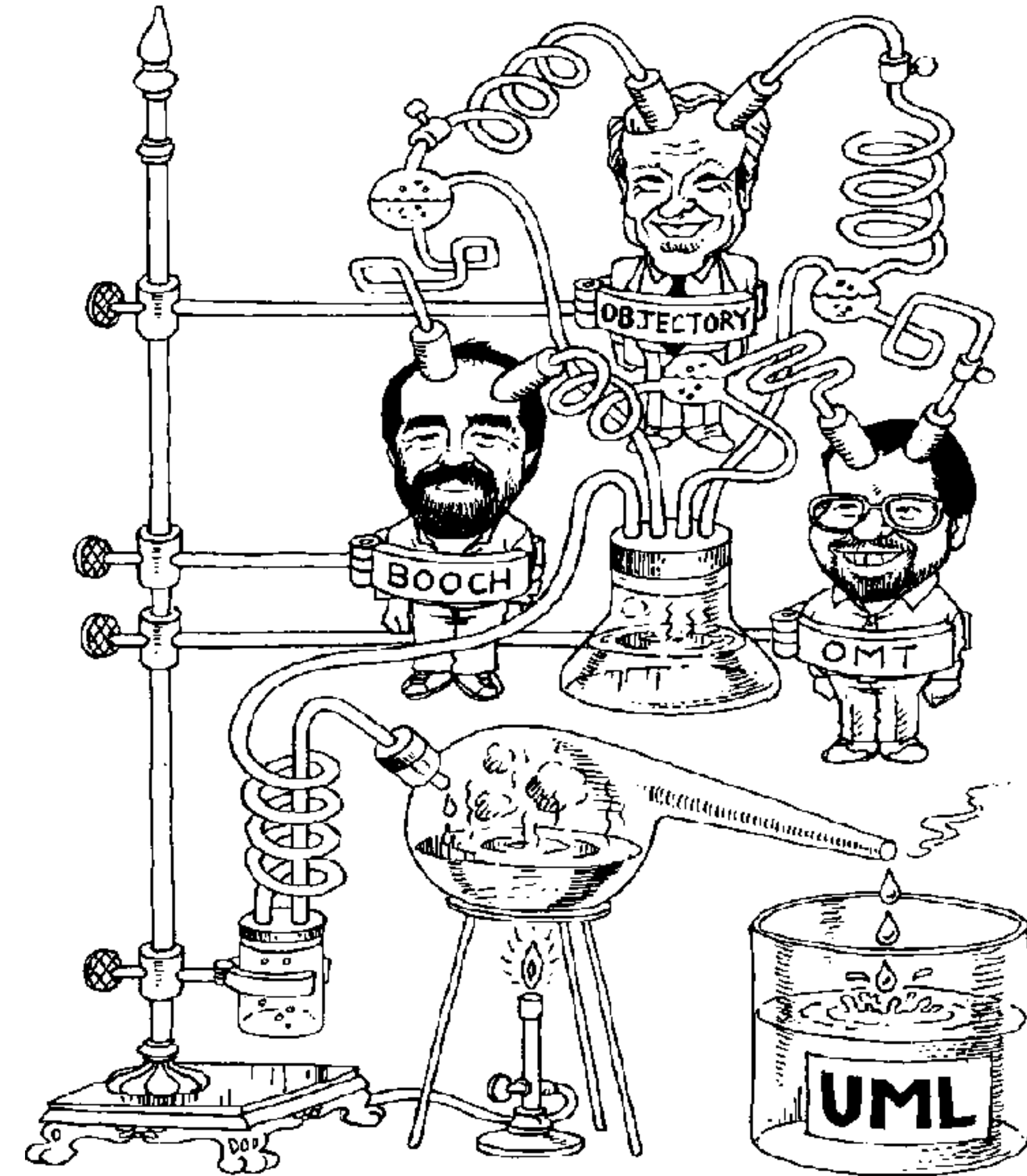
- Um sistema computacional é, de modo geral, excessivamente complexo
- Necessário decompô-lo em pedaços compreensíveis
- Criação de **diagramas** auxiliam no entendimento do problema
- **Linguagem única** que permite a todos os desenvolvedores entender quais objetos fazem parte do sistema e como eles se comunicam



- ◎ A UML surgiu da união de outras três linguagens de modelagem
 - O método de Booch (**Grady Booch**, Rational Software Corporation)
 - O método OMT (ObjectModelingTechnique, **Ivar Jacobson**, **Objectory**)
 - Método OOSE (Object-OrientedSoftware Engineering, **James Rumbaugh**, **General Eletrics**)
- ◎ Até meados da década de 90, estas eram as três linguagens de modelagem mais populares entre os profissionais de ES.



- Em meados da década de 90, os criadores destas três linguagens se reuniram para criar uma **linguagem unificada**, mais concreta e madura





- O objetivo da UML é fornecer múltiplas visões do sistema que se deseja modelar
- Estas várias visões são representadas pelos diferentes diagramas UML
- Cada diagrama analisa o sistema sob um determinado aspecto
 - É possível ter enfoques mais amplos (externos) ou mais específicos (internos)

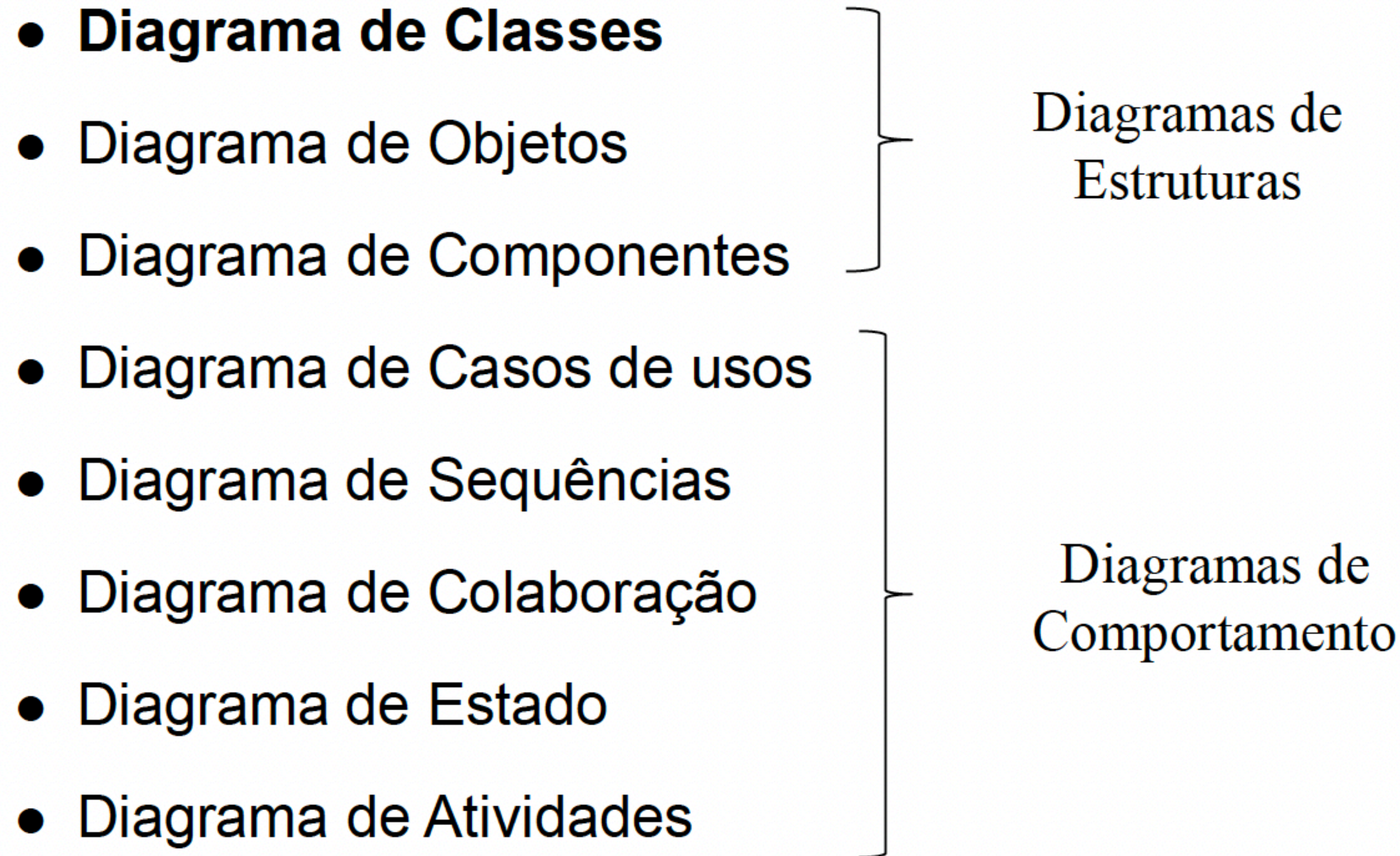


● Perdas

- Maior trabalho na modelagem
 - Mais tempo gasto

● Ganhos

- Menos trabalho na construção (implementação)
 - A solução está pronta
 - Menos tempo gasto
- Os problemas são encontrados em tempo hábil para sua solução
- As dúvidas são sanadas mais cedo e são levantadas em sua totalidade

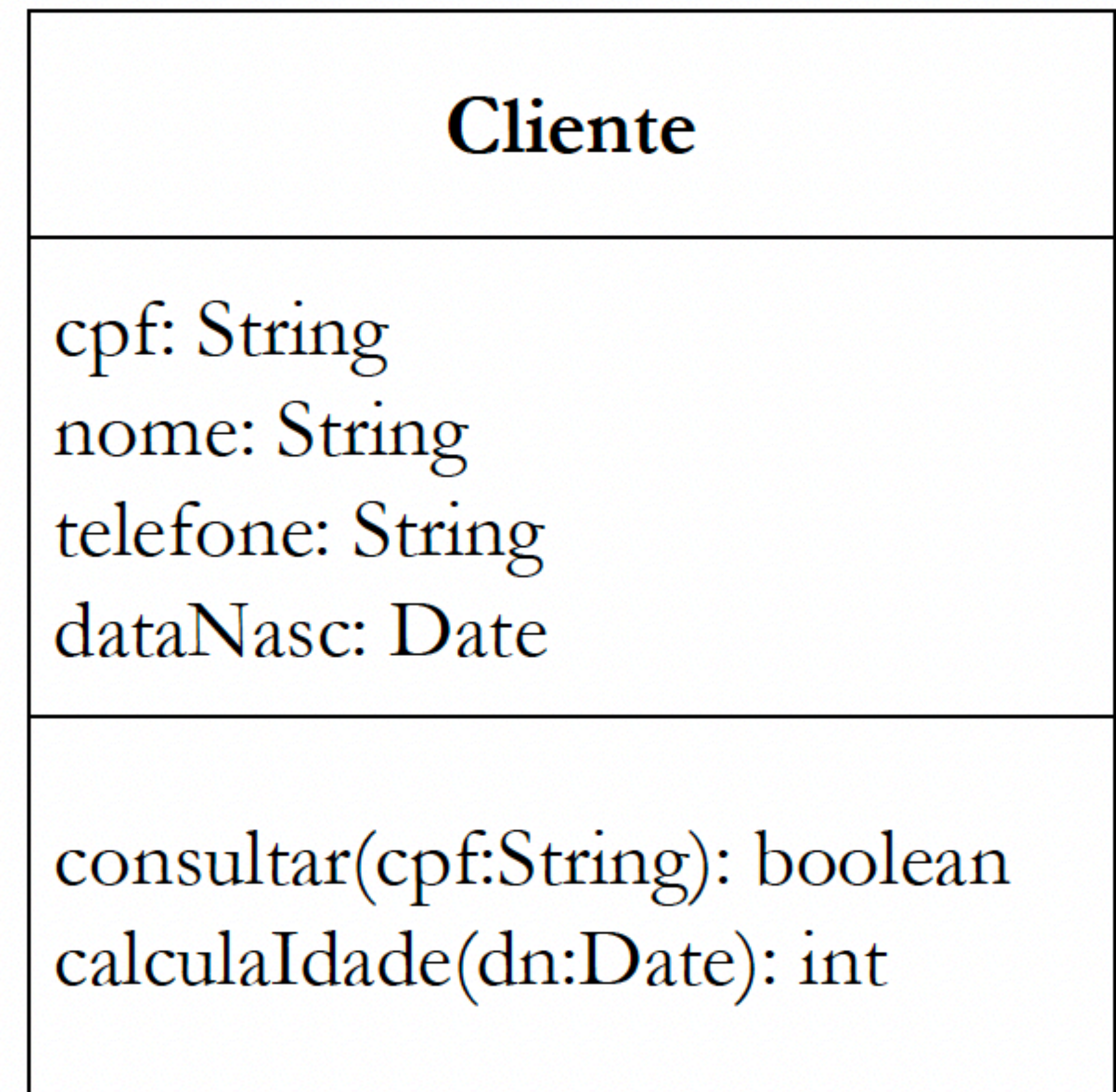




- O diagrama de classes é um dos mais importantes e mais utilizados da UML
- Representação das principais classes
 - Atributos e Métodos
- Relacionamento entre as classes
- Uma visão **estática** do sistema



- Na UML, uma classe possui a notação de um retângulo dividido em três partes
 - Nome da classe
 - Atributos da classe
 - Métodos da classe





● Representação de atributos

visibilidade nome : tipo = valor inicial {propriedades}

- **Visibilidade:** public (+), private (-), protected (#)
- **Tipo do atributo:** int, double, String, Date, ...
- **Valor inicial:** definido no momento da criação do objeto
- **Propriedades:** final, estatic, ...
- Exemplos:

```
-nomeFunc:String = null  
+ PI:double = 3.1415 {final}
```



● Representação de métodos

visibilidade nome(tipo) : tipo {propriedades}

- **Visibilidade:** public (+), private (-), protected (#)
- **Tipo do atributo:** int, double, String, Date, ...
- **Tipo de retorno:** int, double, String, Date, ...
- **Propriedades:** final, abstract, ...

- Exemplos:

```
+ getName() : String {abstract}
    + calcArea(Shape) : double
+ pow(double, double) : double {final}
```

Relacionamento Entre Classes

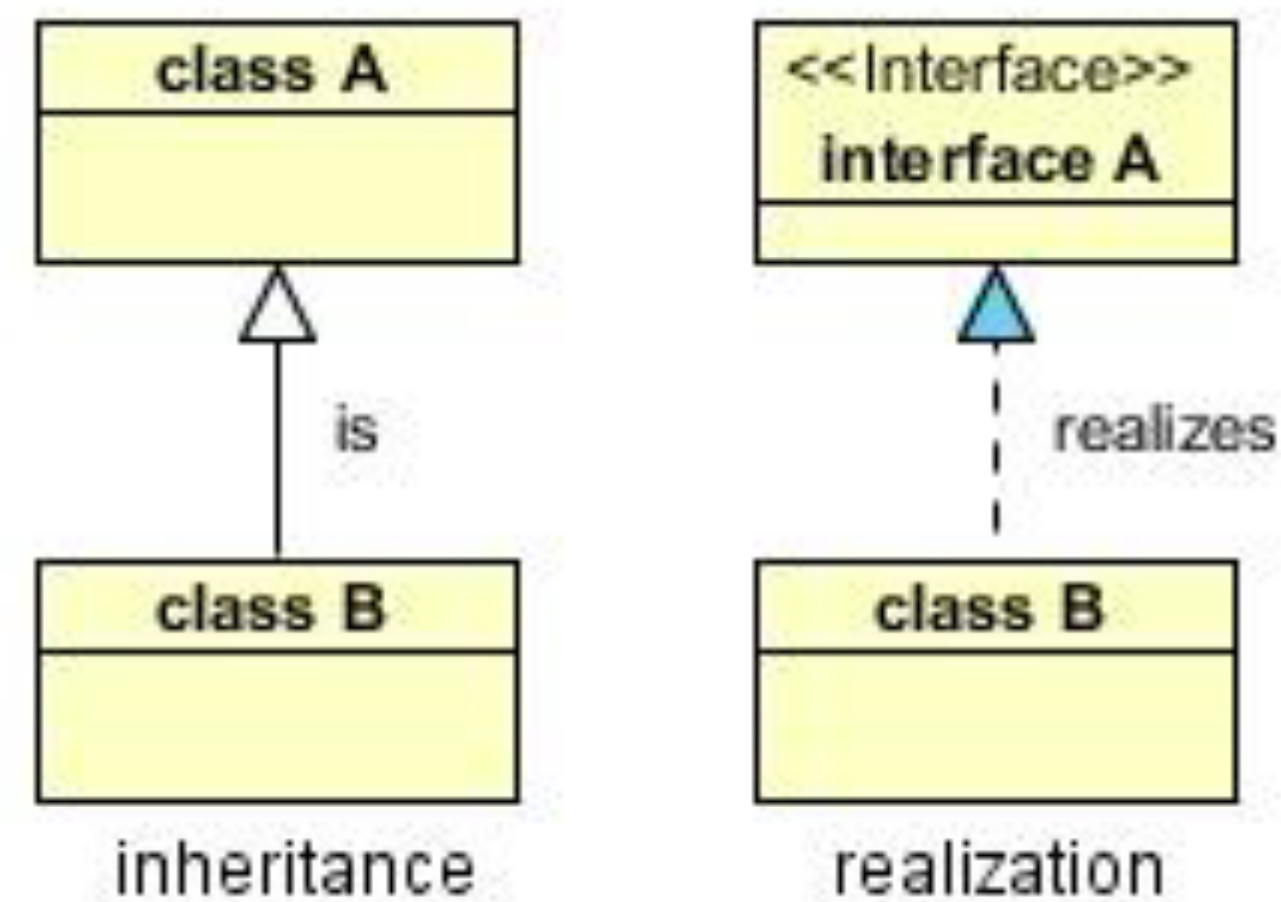
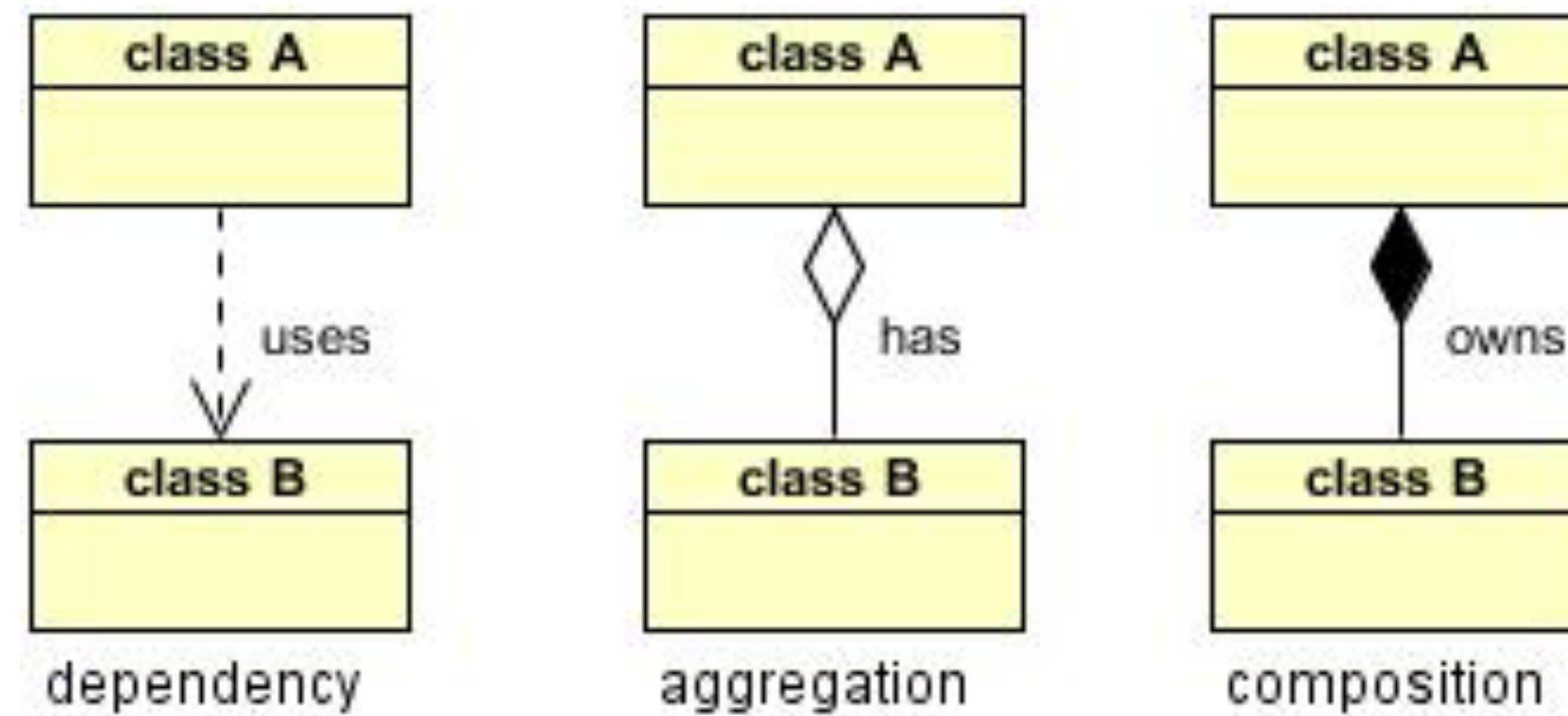


- Em UML é possível representar o relacionamento entre as classes
- Vamos abordar as principais representações
 - Tipos de conexões
 - É uma parte do diagrama de classes



- ◎ Generalização (herança)
 - “é um”
- ◎ Implementação (realização)
 - Aplicada para interfaces
- ◎ Associação (dependência)
 - “usa”
- ◎ Agregação
 - “é parte de” (possui)
 - Objeto ainda faz sentido mesmo sem a existência da agregação
- ◎ Composição
 - “é parte essencial de” (é dono de)
 - Objeto não faz sentido sem a composição

Relacionamento entre classes



Generalização



- Representa relacionamentos do tipo “**é um**”

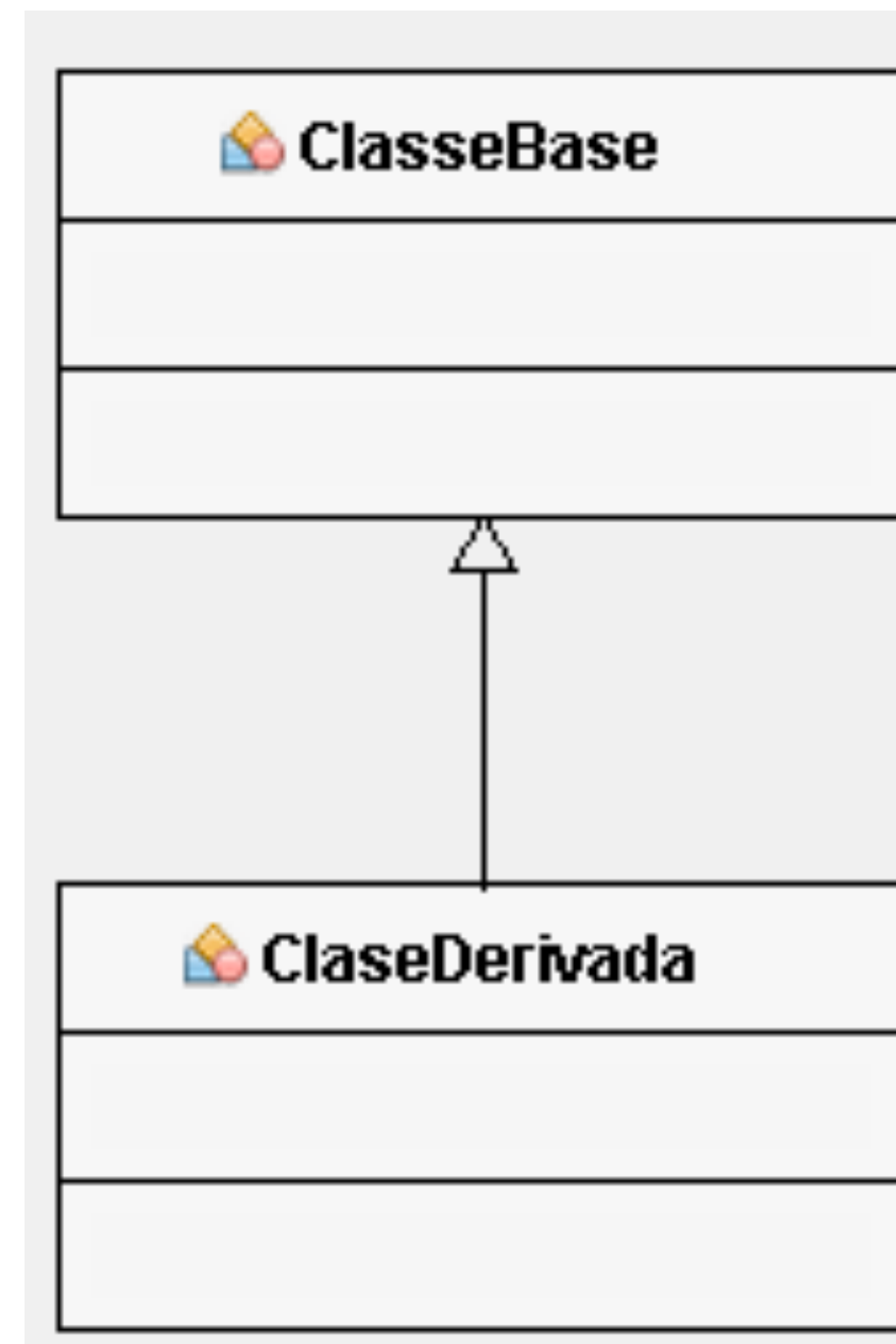
- Herança
- Ex: um cachorro **é um** mamífero

- Generalização/especialização

- A partir de duas ou mais classes, abstrai-se uma classe mais genérica
- De uma classe geral, deriva-se uma mais específica
- Sub-classes possuem todas as propriedades das superclasses
- Deve existir pelo menos uma propriedade que distingue duas classes especializadas
 - Caso contrário, não há necessidade

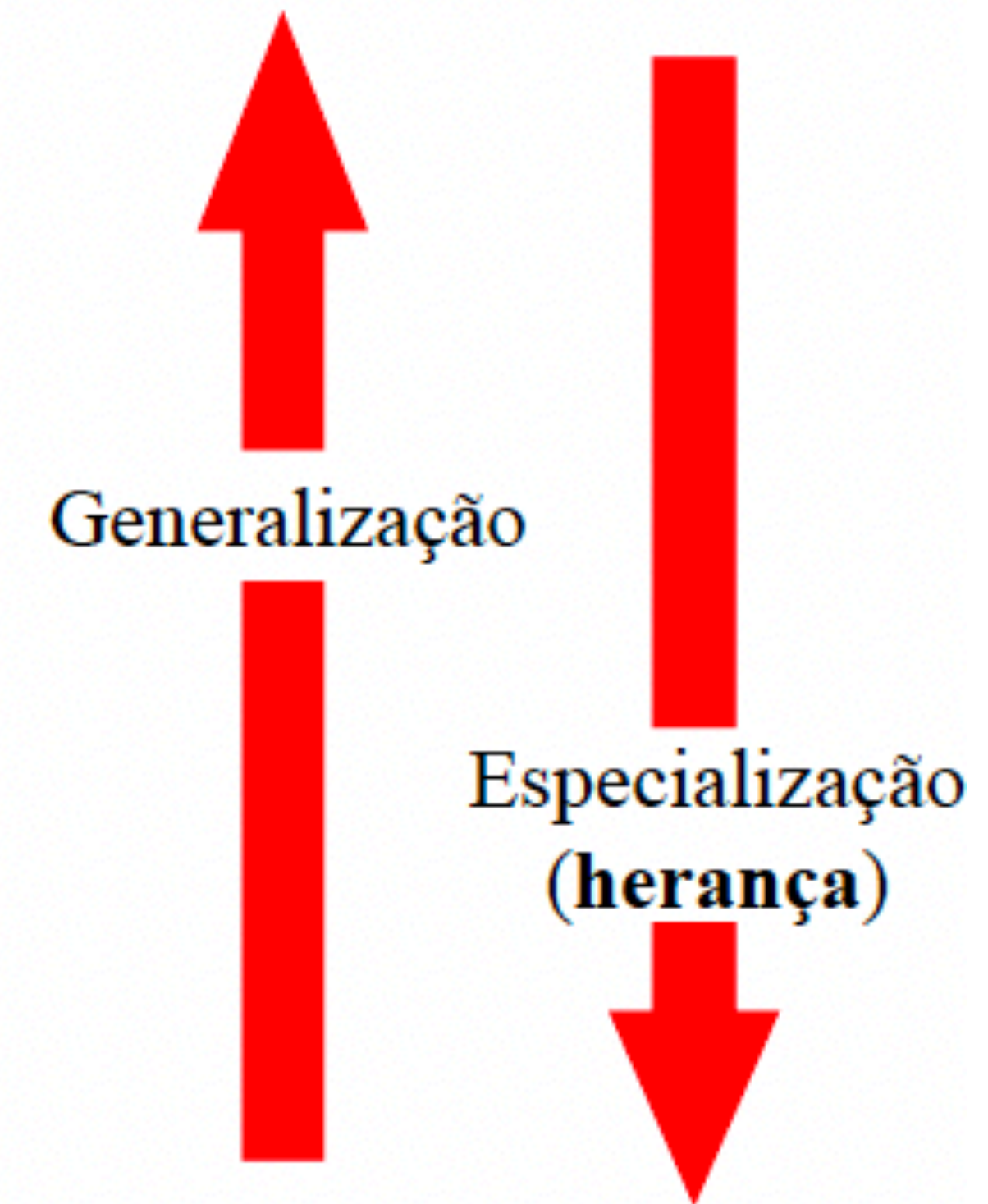
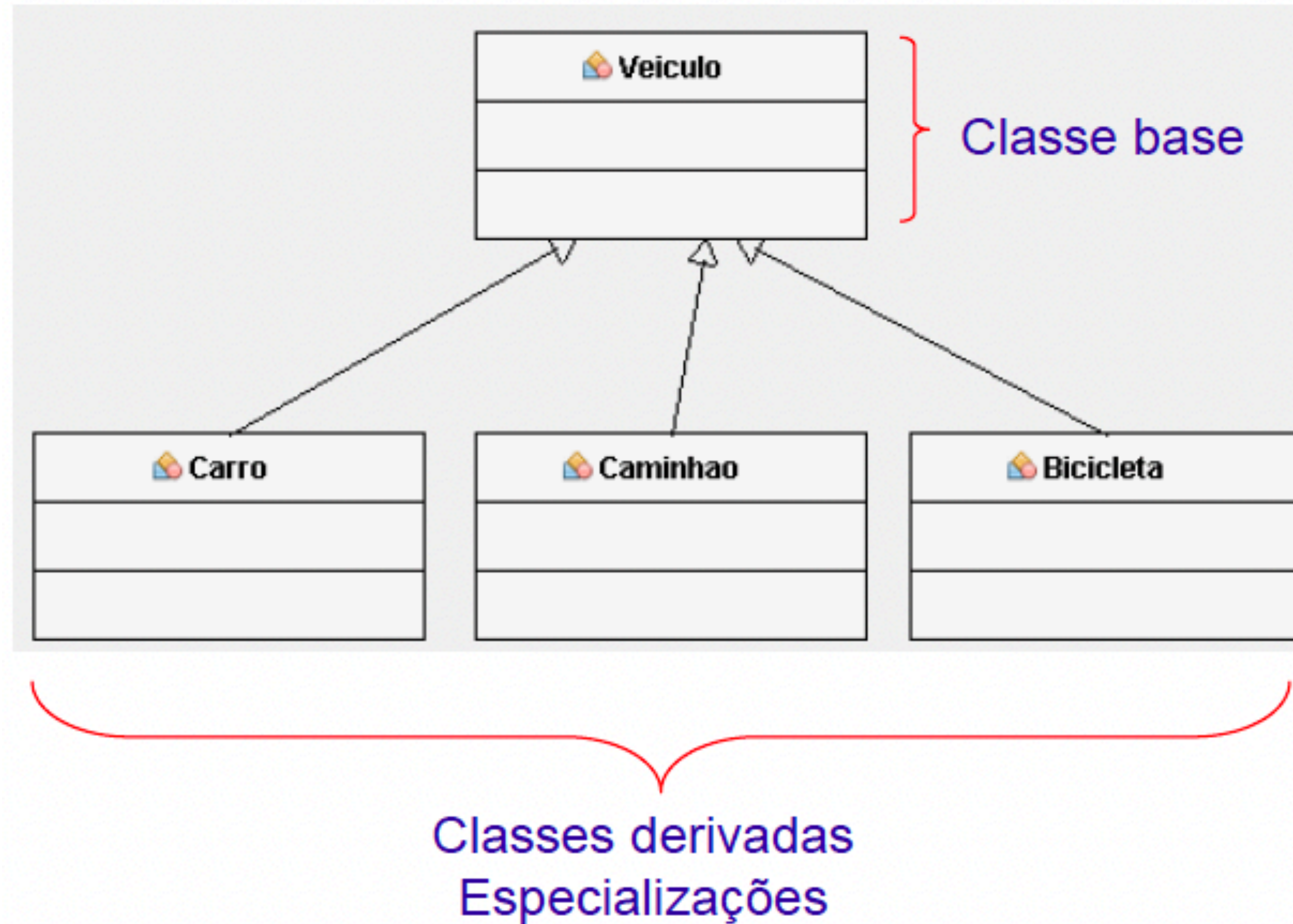


- No diagrama de classes
 - A generalização é representada com uma seta do lado da classe mais geral (classe base)



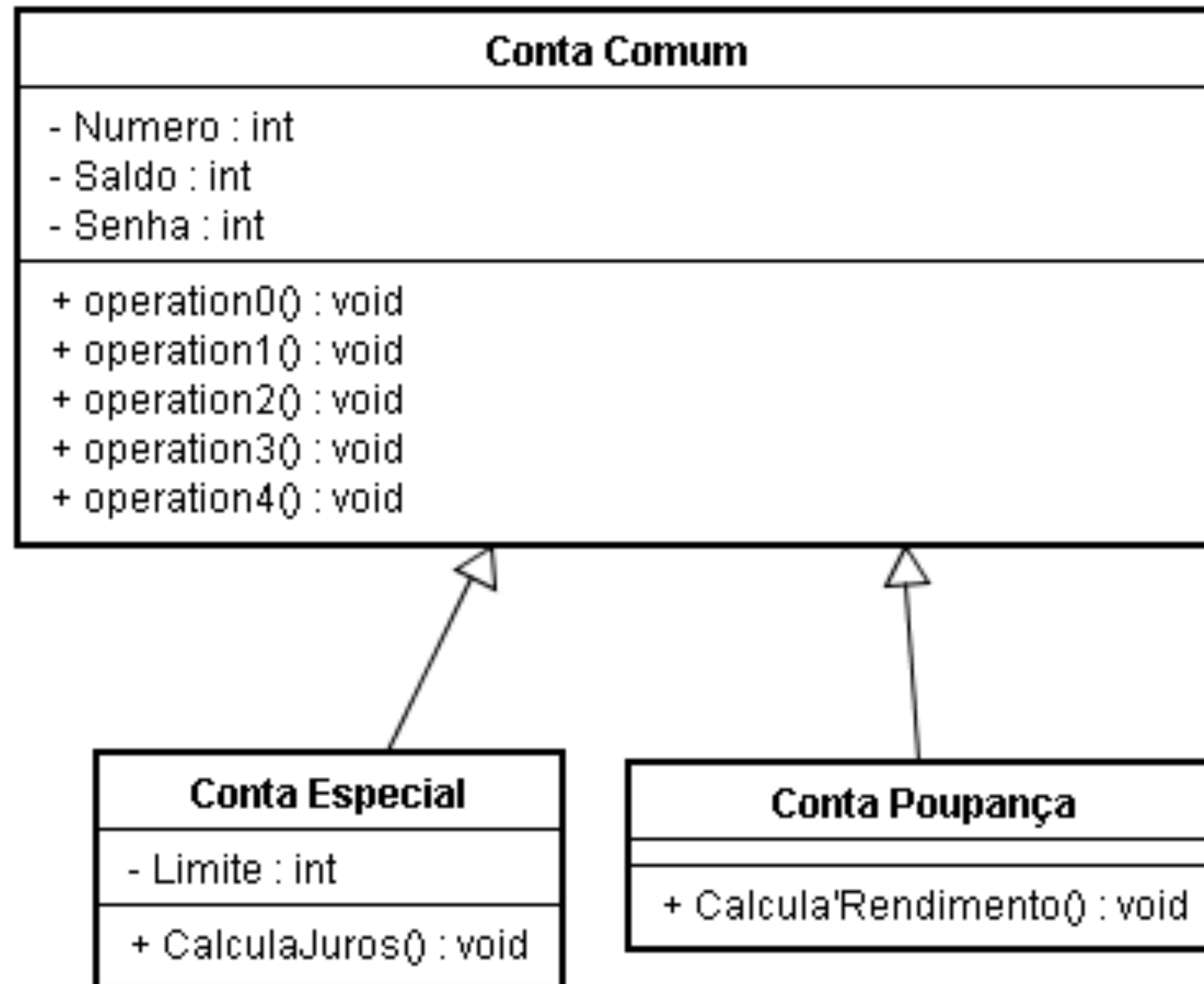


● Exemplo





● Exemplo

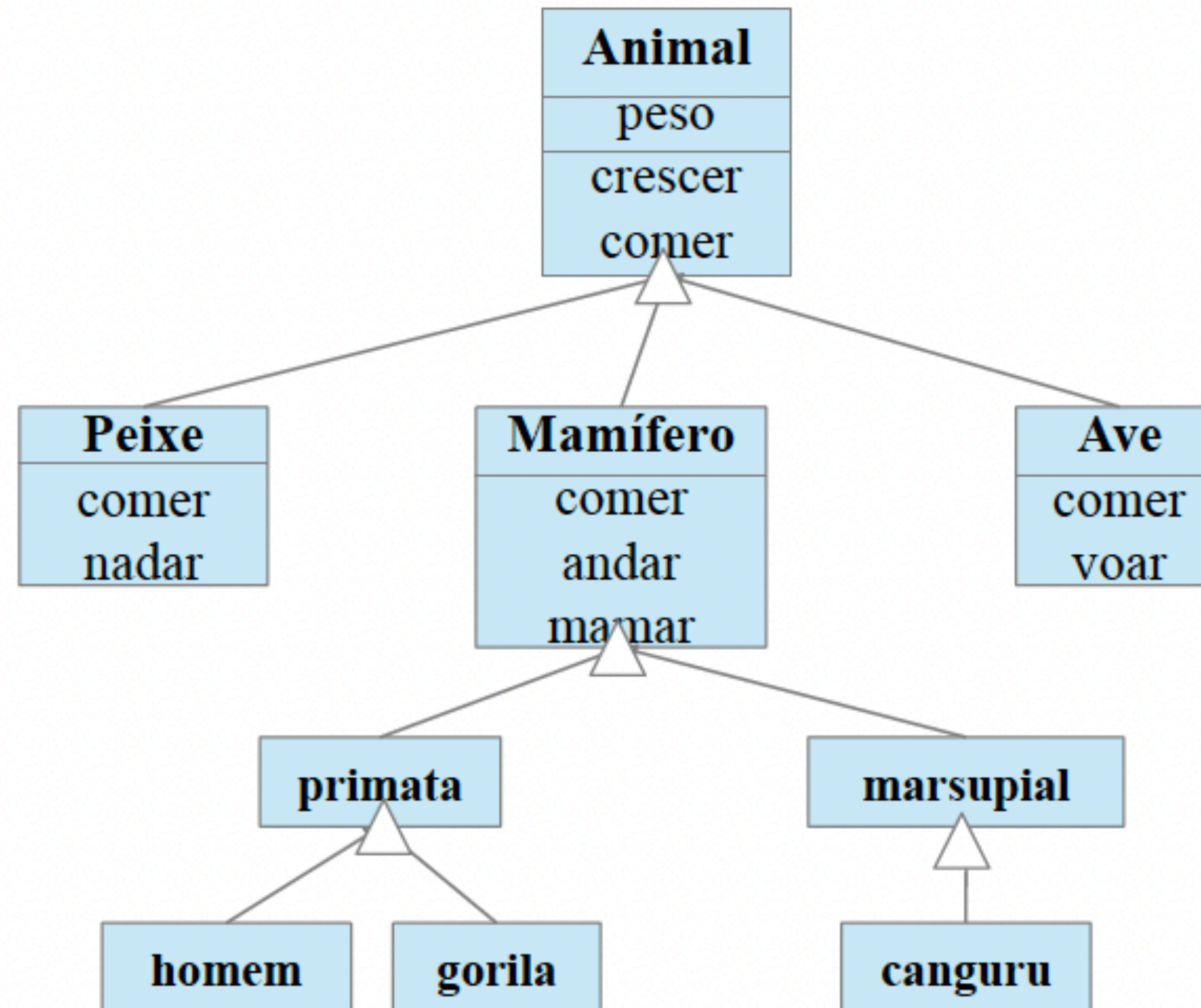




- Permite organizar as classes hierarquicamente
- Técnica de reutilização de software
 - Novas classes são criadas a partir de classes existentes, absorvendo seus atributos e comportamentos (métodos)
 - Recebe novos recursos posteriormente

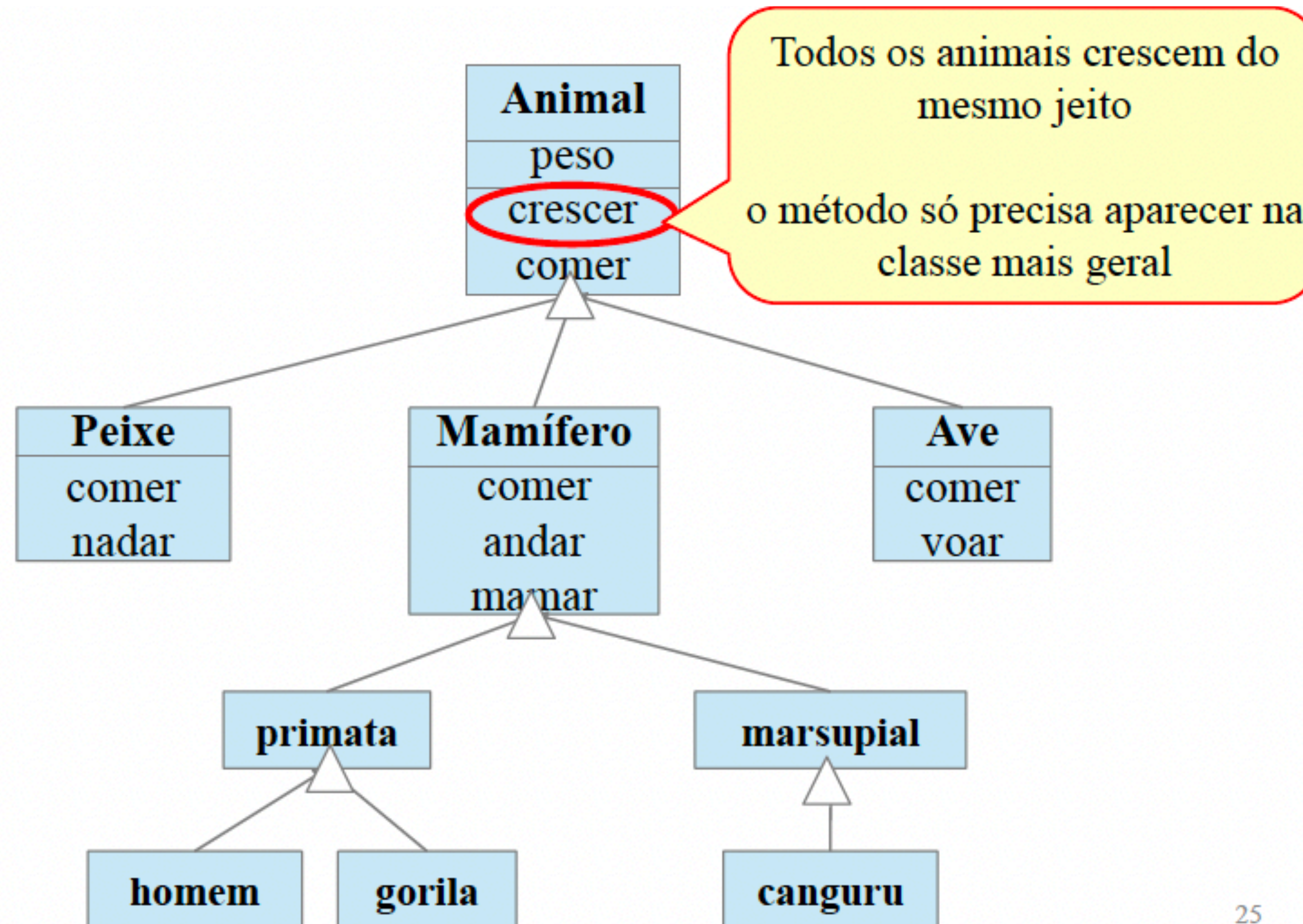


● Exemplo de hierarquia de classes



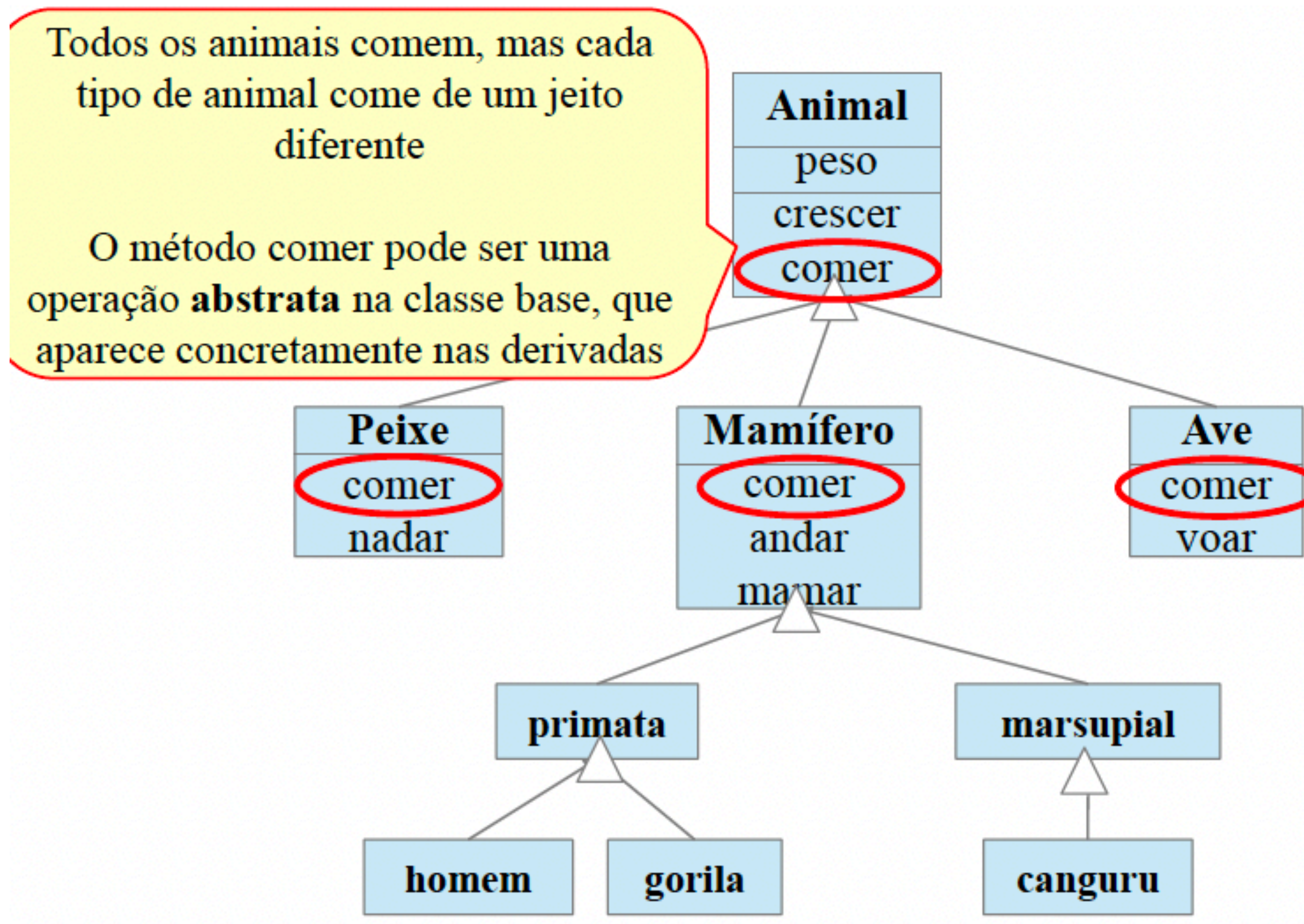


● Exemplo de hierarquia de classes





● Exemplo de hierarquia de classes



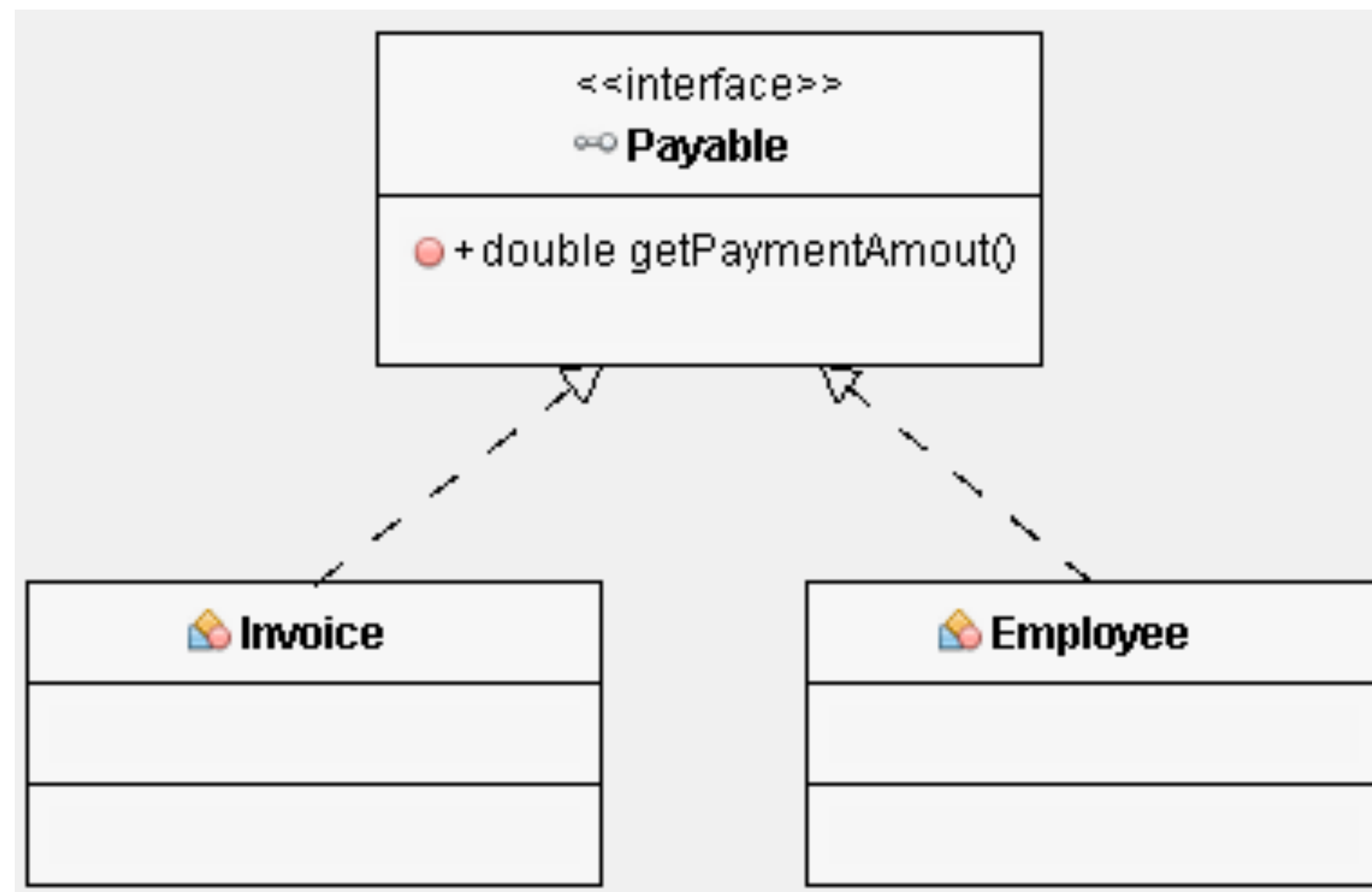
Implementação



- ☉ Interfaces estabelecem um contrato entre os objetos
 - Definição dos métodos pertencentes àquele contrato
- ☉ Interfaces não podem ser instanciadas
 - Não são classes comuns
- ☉ Em classes, podemos usar **herança**
- ☉ Em interfaces, utiliza-se a **implementação**

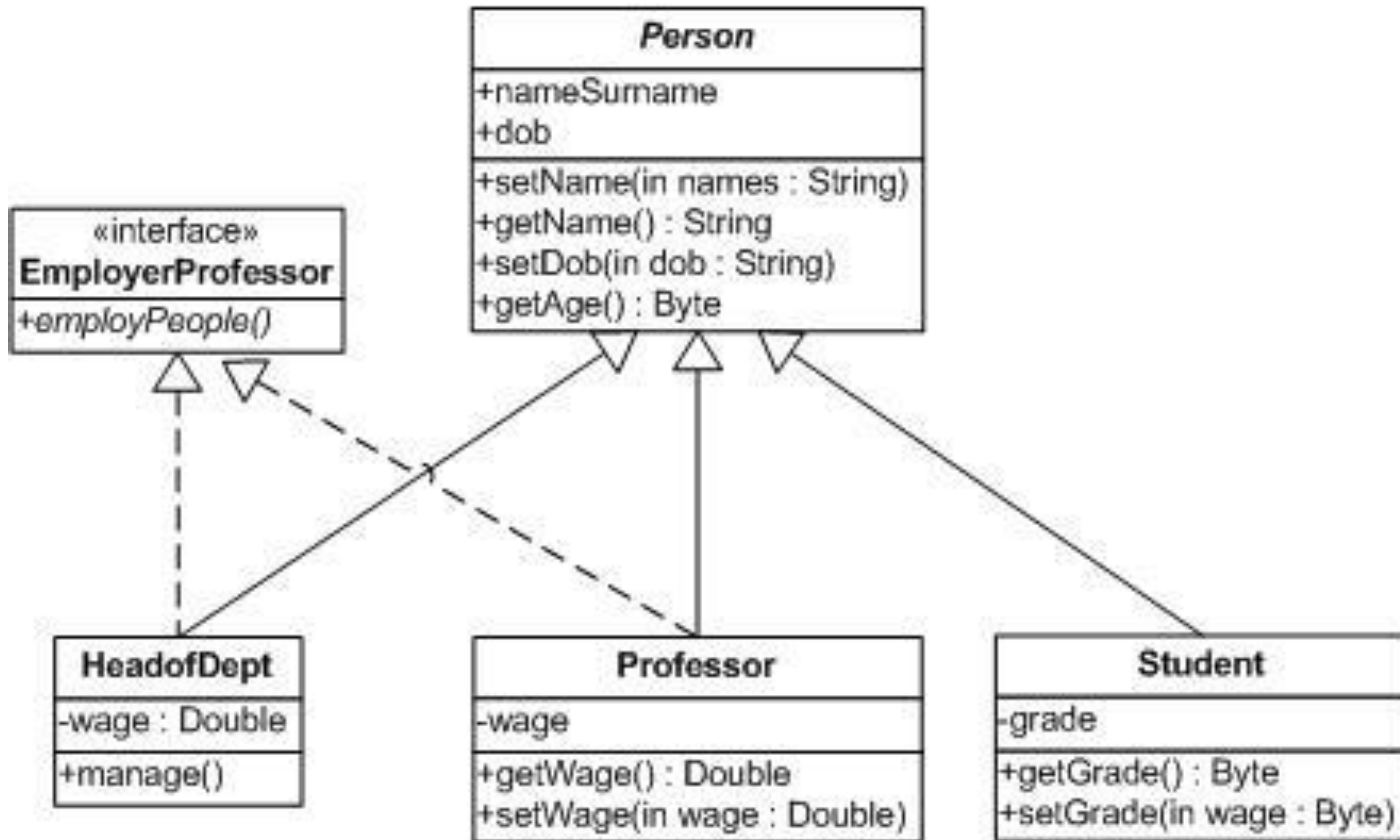


- Em UML, interfaces são definidas de forma similar às classes
 - Diferenciadas com uma marcação de *interface*
 - Implementação é parecida com a herança





- Quando uma classe **herda** outra classe, a implementação dos métodos é herdada
- Quando uma classe **implementa** uma interface, os métodos definidos na interface precisam ser implementados
 - Em geral, não há implementação em uma interface, só definição
 - Todos os métodos da interface precisam necessariamente ser escritos pela classe que implementa a interface



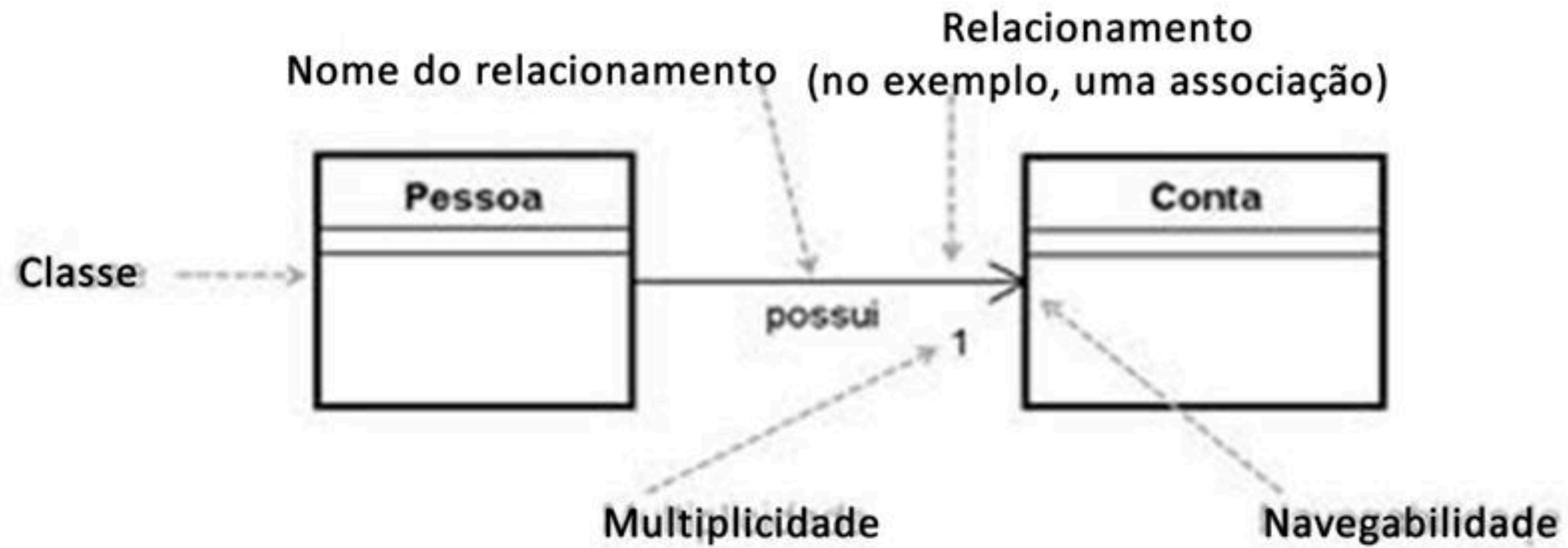
Relacionamentos



- © Antes de continuar com **Agregação**, **Composição** e **Associação** vamos fazer um parênteses e falar um pouco sobre propriedades dos relacionamentos

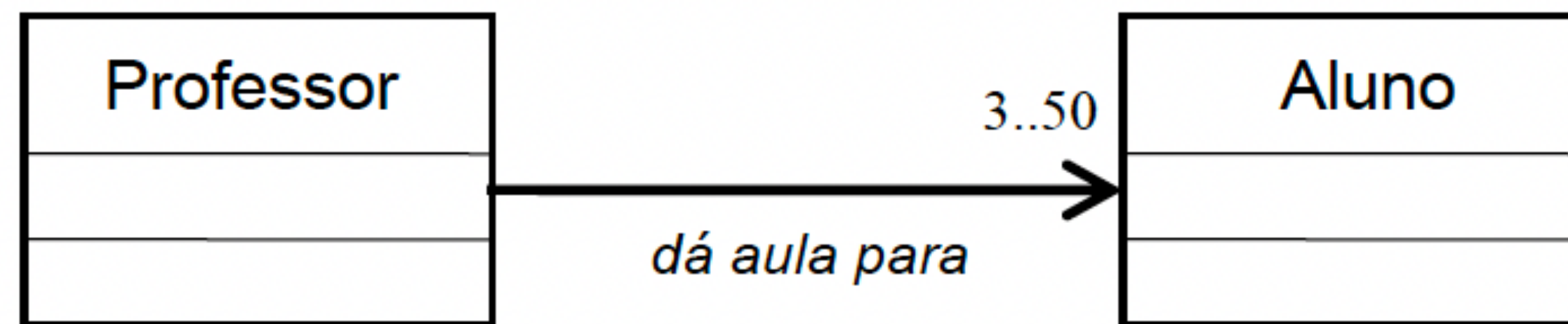


- ◎ Os relacionamentos são caracterizados por
 - Nome
 - Descrição do relacionamento
 - Em geral usa-se um verbo
 - Faz, tem, possui
 - Navegabilidade
 - Indicada por uma seta no fim do relacionamento
 - Uni (uma flecha) ou bidirecional (sem flechas/duas flechas)
 - Multiplicidade
 - Quantidade de elementos que cada relacionamento pode assumir
 - 0..1, 0..*, 1, 1..*, 2, 3..7



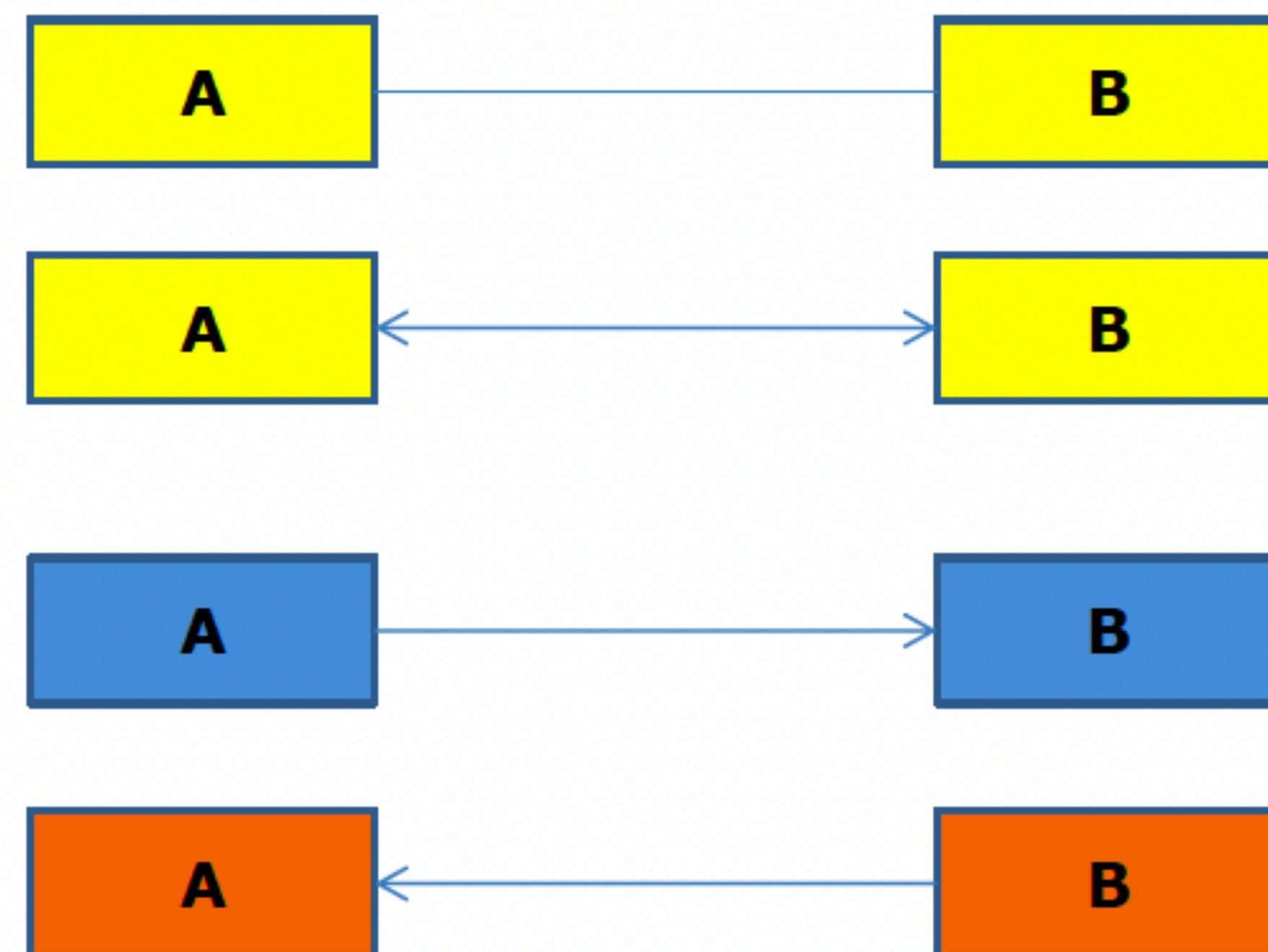


- Nomear um relacionamento facilita o entendimento
- Nome do relacionamento (rótulo) é colocado ao longo da linha de associação



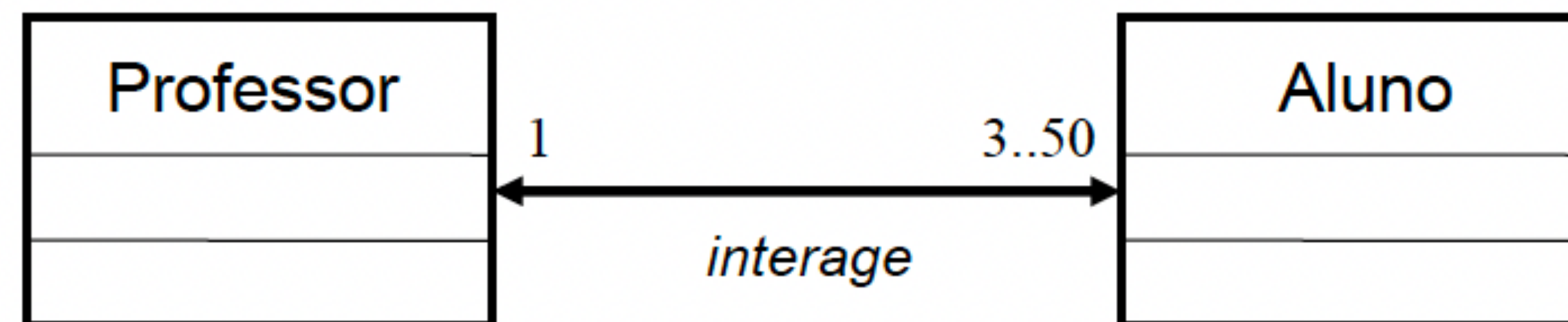


- Navegabilidade indica a direcionalidade com que as classes se relacionam
- Ambas as classes se relacionam (sabem da existência uma da outra)
- **B** não sabe da existência de **A**
- **A** não sabe da existência de **B**





- Multiplicidade é o **número de instâncias** de uma classe relacionada com uma ou mais instâncias de outra classe
- Exemplo: Professor e Aluno
 - Cada Professor pode interagir com 3 a 50 Alunos
 - Cada Aluno pode interagir com apenas um Professor
 - Pensando em um único curso





Muitos

*

Exatamente um

1

Zero ou mais

0..*

Um ou mais

1..*

Zero ou um

0..1

Faixa especificada

2..4



● Exemplos

- Uma mesa de restaurante pode ter vários ou nenhum pedido
 - $*..0$
- Uma cotação pode incluir no mínimo 1 e até muitos (*) itens cotados
 - $1..*$
- Uma casa pode ter de 0 a 3 funcionários
 - $0..3$

Associação



- É a forma mais fraca de relacionamento entre classes
 - As classes que participam desse relacionamento são **independentes**
 - São representadas como linhas conectando as classes participantes
 - Podem ter um nome identificando a associação
 - Podem ter uma seta junto ao nome indicando que a associação somente pode ser utilizada em uma única direção (o mais usual e adequado)
 - Representa relacionamentos “**usa um**”
 - Pessoa **usa um** Carro



- Na implementação
 - ObjetoA **usa** ObjetoB quando o ObjetoA **chama um método público** do ObjetoB
- Associação simples também é chamada de **dependência**
- Diagramas de dependência são os primeiros diagramas usado para compreender um código que não é seu
- É crucial para entender como diferentes partes de um sistema interagem entre si, sem criar um acoplamento forte ou relações de posse entre os objeto
- É comum em cenários onde um objeto precisa realizar uma operação que é delegada a outro objeto, mas sem manter uma ligação estreita com ele



● Vamos ilustrar essa ideia com um exemplo em Java, usando dois objetos simples:

- **Impressora** (ObjetoB) e **Documento** (ObjetoA). Neste caso, **Documento** usa **Impressora** para realizar a operação de imprimir, mas não mantém uma referência permanente a Impressora
- Isso ilustra uma associação simples ou dependência

```
public class Impressora {
    public void imprimir(String conteudo) {
        System.out.println("Imprimindo: " + conteudo);
    }
}

public class Documento {
    private String conteudo;

    public Documento(String conteudo) {
        this.conteudo = conteudo;
    }

    public void imprimirDocumento(Impressora impressora) {
        impressora.imprimir(this.conteudo);
    }
}

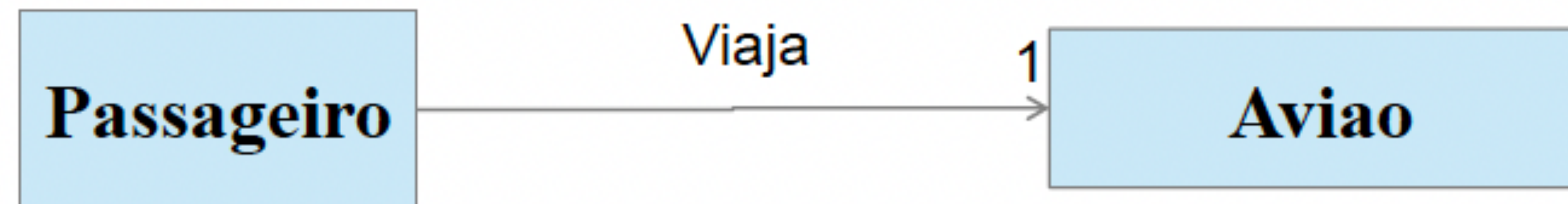
public class Main {
    public static void main(String[] args) {
        Impressora impressora = new Impressora();
        Documento documento = new Documento("Olá, mundo!");

        documento.imprimirDocumento(impressora);
    }
}
```



● Exemplo

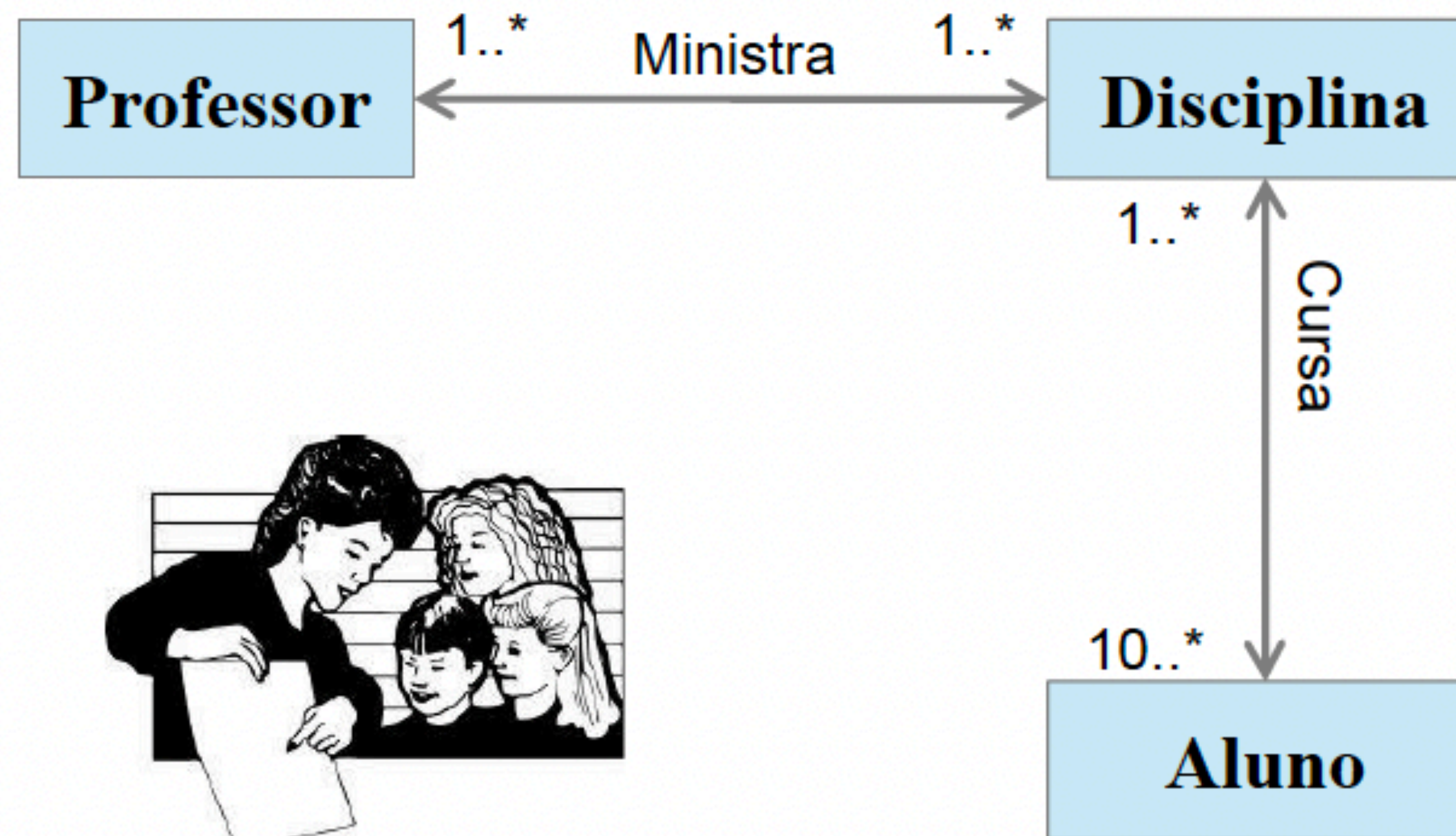
- Um **Passageiro** pode viajar para qualquer lugar, dependendo de qual **Avião** ele entrar
- Para que um **Passageiro** viaje, ele precisa apenas de uma indicação de qual **Avião** ele deve entrar. Ele não precisa ter como parte de sua informação (atributo) a referência a um **Avião**.



- Leitura unidirecional
 - Um **Passageiro** viaja em um **Avião**

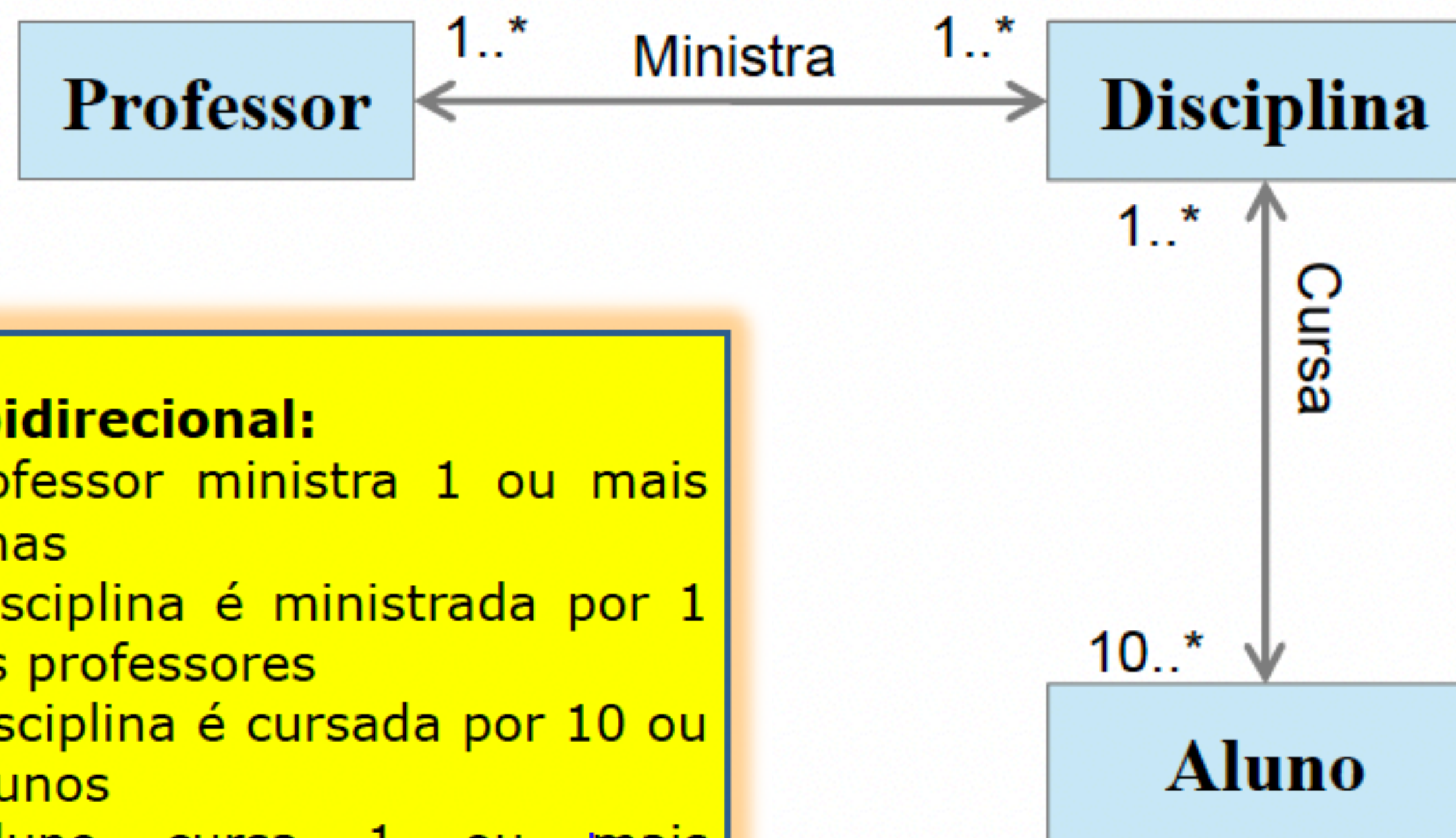


● Exemplo bidirecional





● Exemplo bidirecional



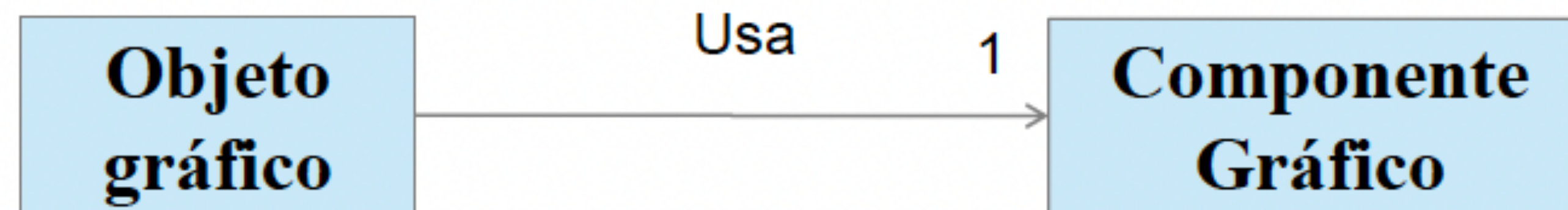
Leitura bidirecional:

- Um professor ministra 1 ou mais disciplinas
- Uma disciplina é ministrada por 1 ou mais professores
- Uma disciplina é cursada por 10 ou mais alunos
- Um aluno cursa 1 ou mais disciplinas



● Outro exemplo

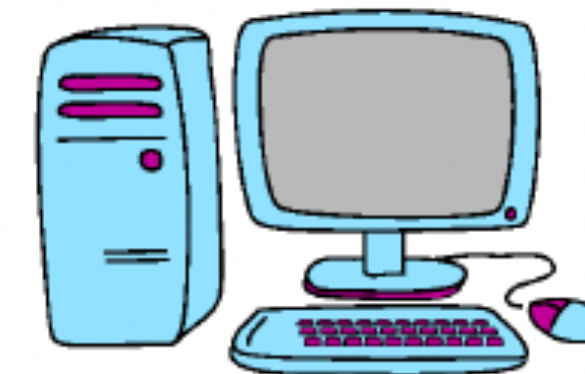
- Imagine um objeto gráfico que se auto-desenha.
- O objeto sabe como se desenhar, mas precisa de acesso a funcionalidade gráficas exclusivas de componentes gráficos do sistema.
- Para se desenhar, o objeto gráfico deve receber como parâmetro um componente gráfico em seu método *autoDesenho(CompGrafico comp)*.
- Ele irá apenas usar a classe CompGráfico, sem contudo ser composto por ela.



Agregação e Composição



- São também formas de associação, mas representam relacionamentos do tipo “**tem um**”
 - Uma classe é **formada por** ou **contém** objetos de outras classes
 - Exemplos
 - Um carro possui rodas
 - Uma árvore é composta de folhas, tronco, raízes, ...
 - Um computador é composto de CPU, memória, teclado, mouse, monitor, ...

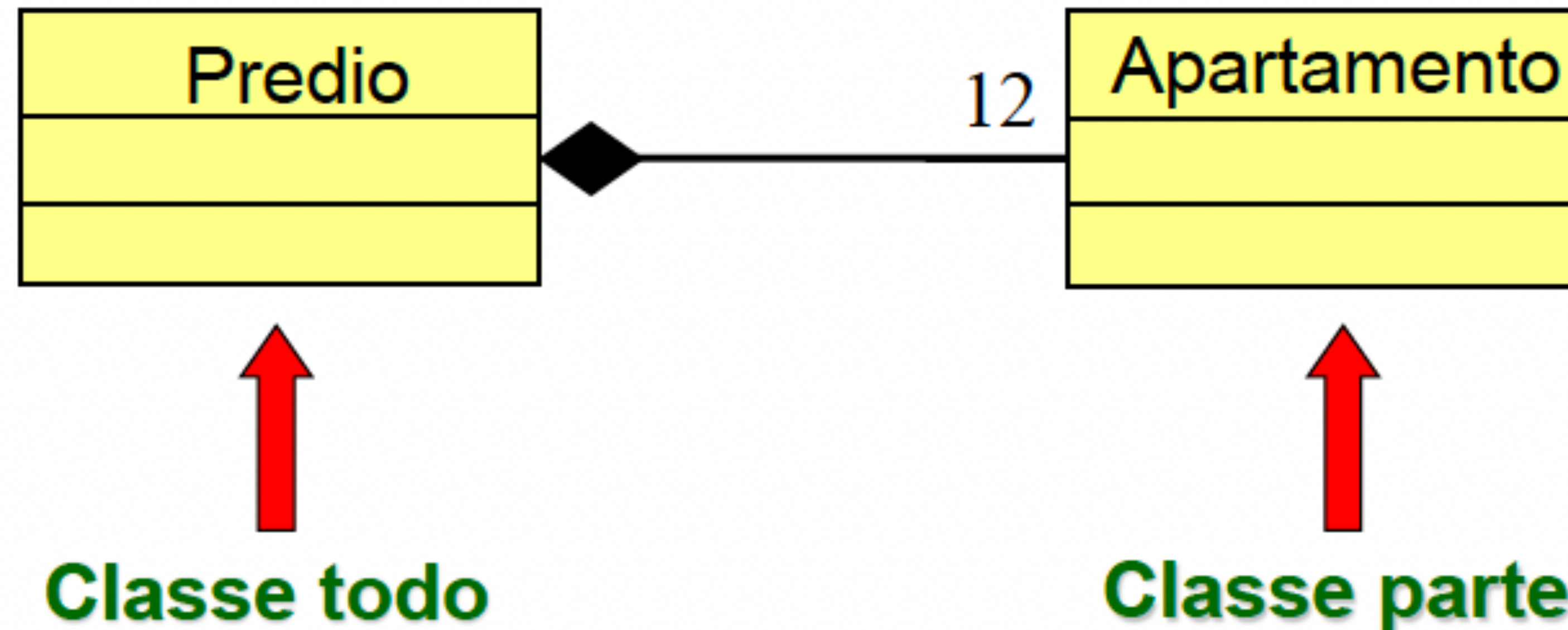




- Classe todo
 - É a classe resultante da agregação/composição
- Classe parte
 - É a classe cujas instâncias formam a agregação/composição
- Exemplo de **composição**: **Predio** e **Apartamento**
 - Um prédio tem apartamentos
 - Classe **Predio**: todo ou agregada
 - Classe Apartamento: parte



- Prédio tem como atributo um conjunto (*array*) de apartamentos
- Se o prédio deixar de existir, os apartamentos também deixam de existir
- Segundo a cardinalidade, um prédio precisa ter obrigatoriamente 12 (exatos) apartamentos





- Na composição, o **todo** é responsável pelo ciclo de vida da **parte**.
- Também se diz que o **todo** é dono da **parte**, e não apenas “possui a **parte**”
- Assim, em composição, a criação da **parte** ocorre no **todo**.



- Agregação é uma forma mais fraca de composição
- **Composição:** relacionamento todo-parte em que as partes não podem existir independentes do todo
 - Se o **todo é destruído**, as **partes são destruídas** também
 - Uma parte pode ser de um todo por vez
- **Agregação:** relacionamento todo-parte que não satisfaz um ou ambos os critérios
 - A **destruição do objeto** não implica a **destruição do objeto parte**
 - Um objeto pode ser parte componente de vários outros objetos



◎ Em termos de implementação, significa que:

- Na composição

- Os objetos das classes parte são criados e gerenciados diretamente dentro da classe que representa o todo
- Os objetos parte não são acessíveis ou manipuláveis de forma independente por entidades externas à classe todo, garantindo uma relação de dependência forte e exclusiva entre o todo e suas partes

- Na agregação

- Classe todo mantém referências aos objetos das classes que representam as partes
- No entanto, esses objetos parte podem ser criados e existir independentemente da classe todo
- Eles podem ser passados à classe todo por meio de parâmetros em métodos ou construtores
- Mesma instância da classe parte seja compartilhada entre diferentes objetos todo ou exista sem estar atrelada a nenhum objeto todo específico



```
class Pagina {
    private String conteudo;

    public Pagina(String conteudo) {
        this.conteudo = conteudo;
    }
    // Métodos da classe Pagina
}

class Livro {
    private List<Pagina> paginas;

    public Livro() {
        paginas = new ArrayList<>();
        // Supondo que um livro seja criado
        // com um número definido de páginas
        for (int i = 0; i < 100; i++) {
            paginas.add(new Pagina("Conteúdo da Página " + (i + 1)));
        }
    }
    // Métodos do Livro
    // Note que não há um método público
    // para adicionar páginas externas
}
```

```
class Pagina {
    // Implementação da classe Pagina,
    // que não faz sentido existir sem Livro
}

class Livro {
    // Livro tem muitas Paginas
    private List<Pagina> paginas;
    public Livro() {
        paginas = new ArrayList<>();
    }

    public void adicionarPagina(Pagina pagina) {
        paginas.add(pagina);
    }

    // Métodos do Livro
}
```



- No diagrama de classes

- Composição

- Associação representada com um losango **sólido** do lado **todo**

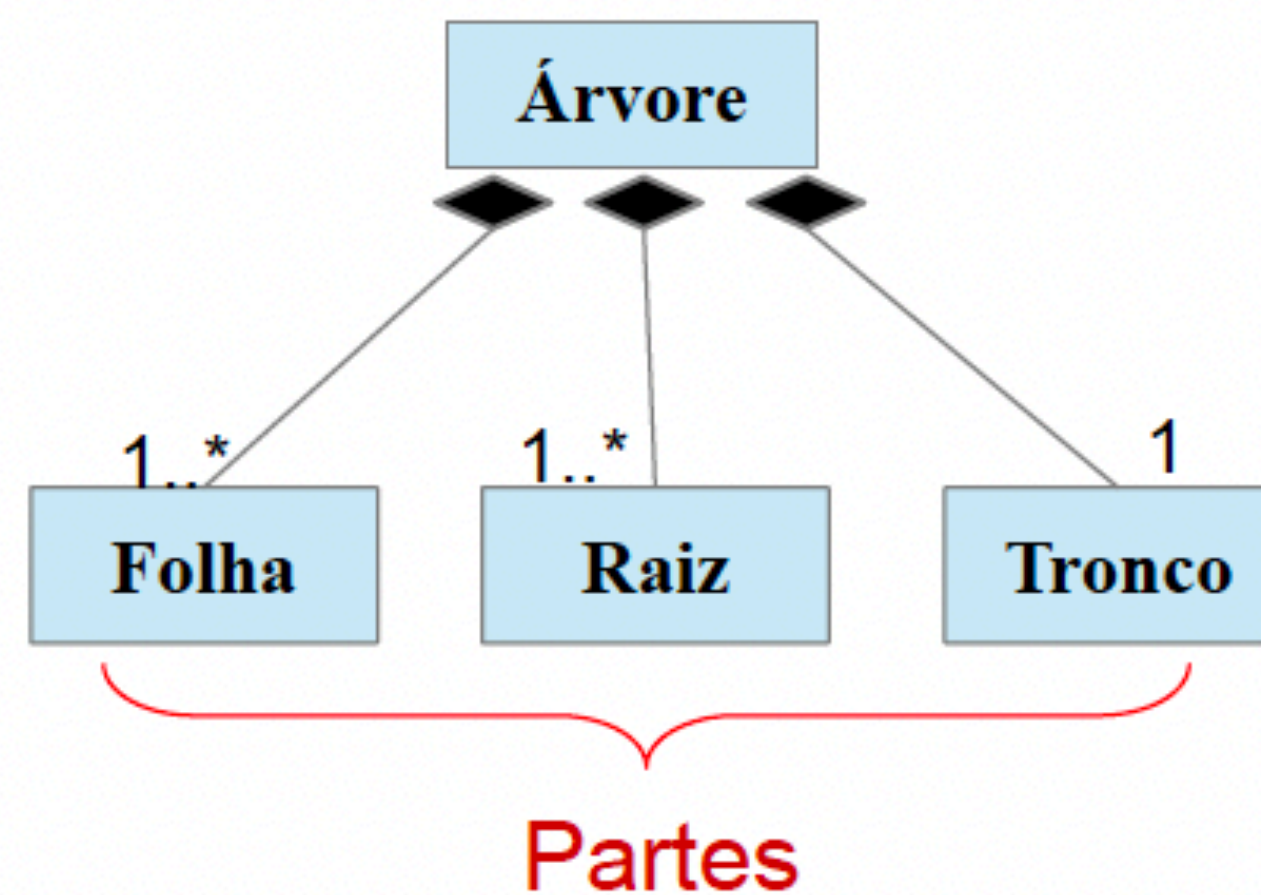


- Agregação

- Associação representada com um losango **sem preenchimento** do lado **todo**



- Não faz sentido que **Folha**, **Raiz** ou **Tronco** existam sem que sejam atributos de uma **Árvore**
 - Neste modelo em particular
- Ainda segundo o diagrama, não pode haver uma **Árvore** sem **Folha**, **Raiz** ou **Tronco** (cardinalidade)

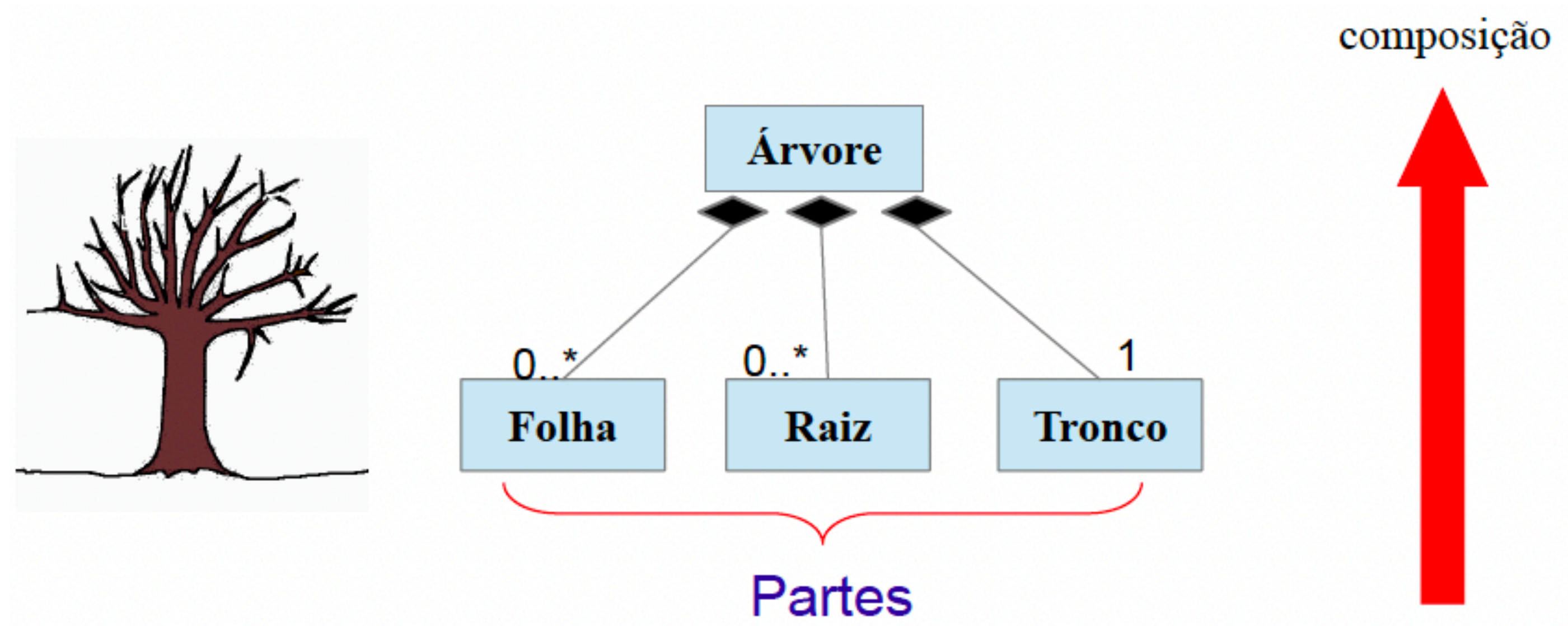


composição

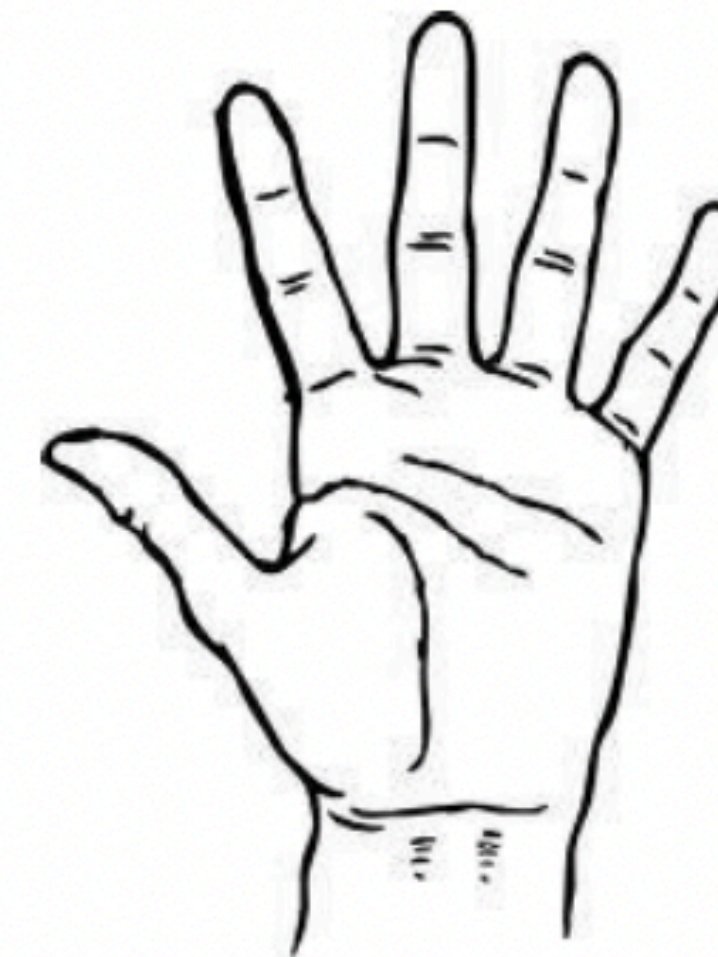
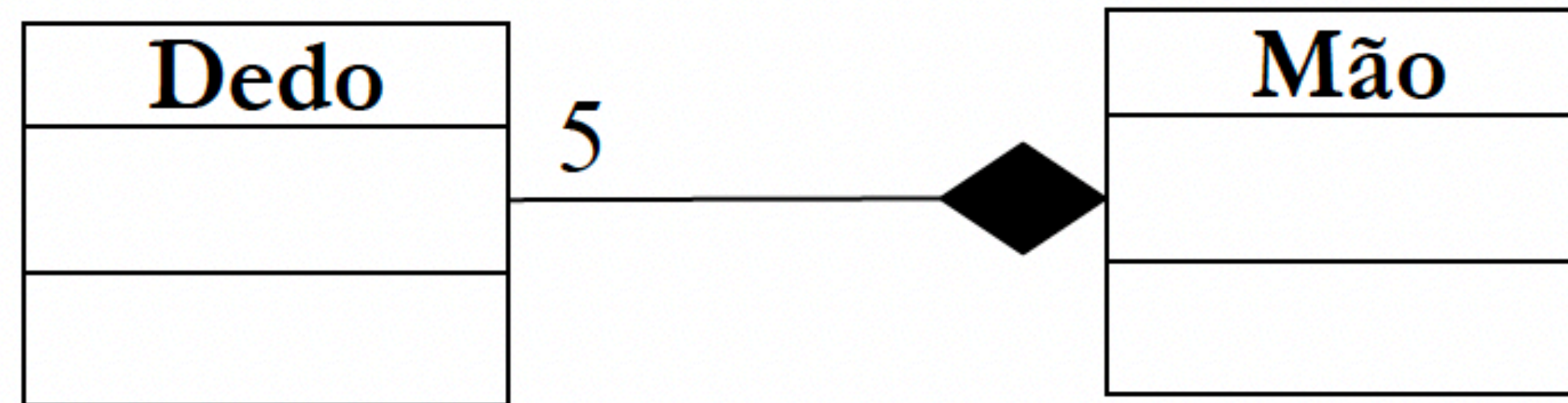




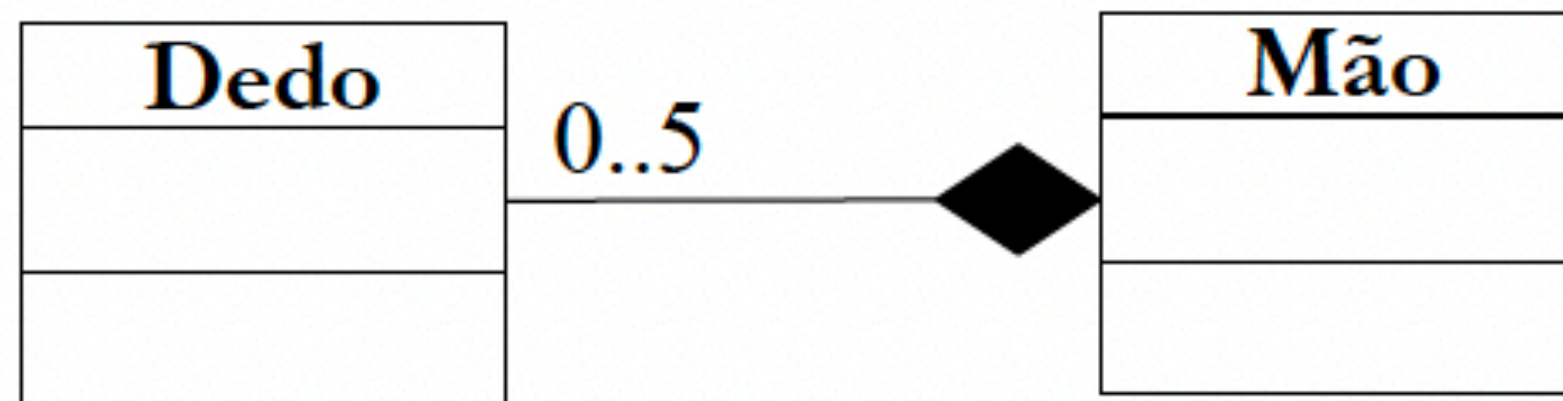
- Agora pode haver uma Árvore sem Folha e sem Raiz, mas não sem Tronco



- Não faz sentido que um **Dedo** exista se não for parte de uma **Mão**
 - Segundo a cardinalidade, não pode haver uma mão sem dedos
 - Todo mão tem exatos 5 Dedos

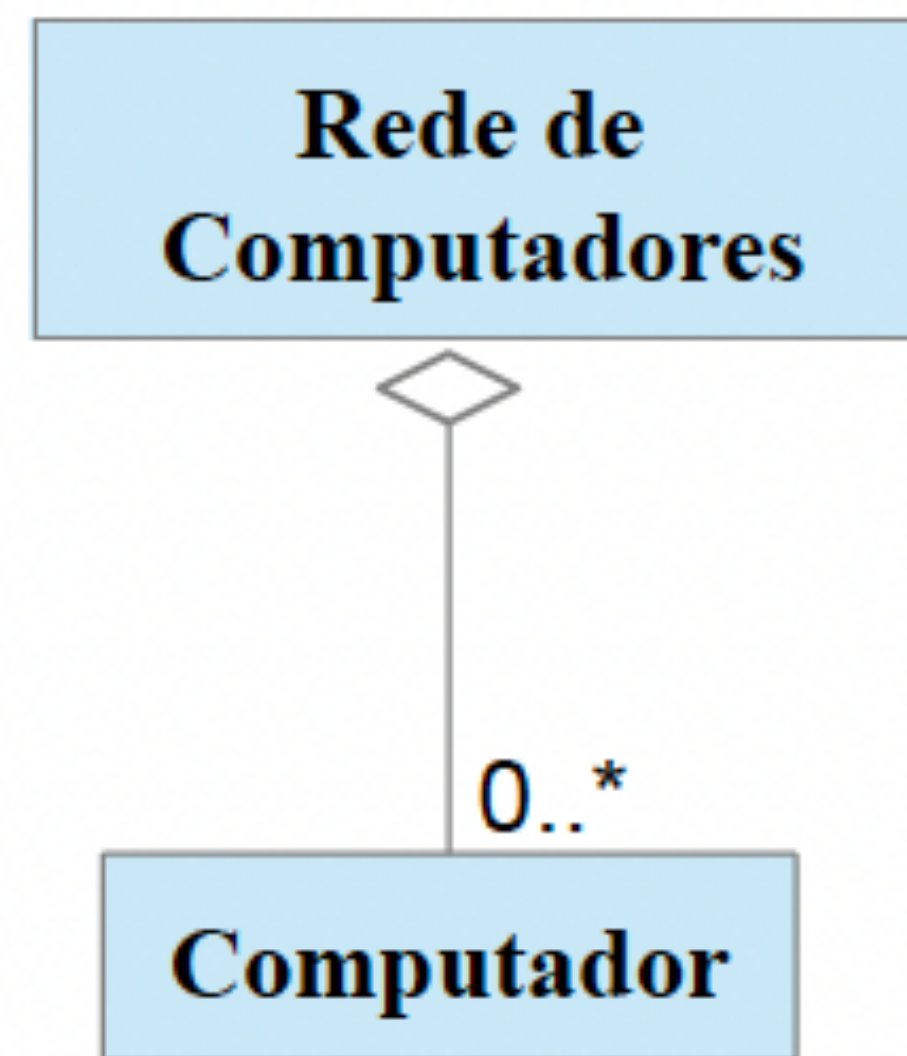


- Não faz sentido que um **Dedo** exista se não for parte de uma **Mão**
 - Na definição de agora, uma mão pode não ter dedos
 - Cardinalidade mínima é 0 e máxima é 5



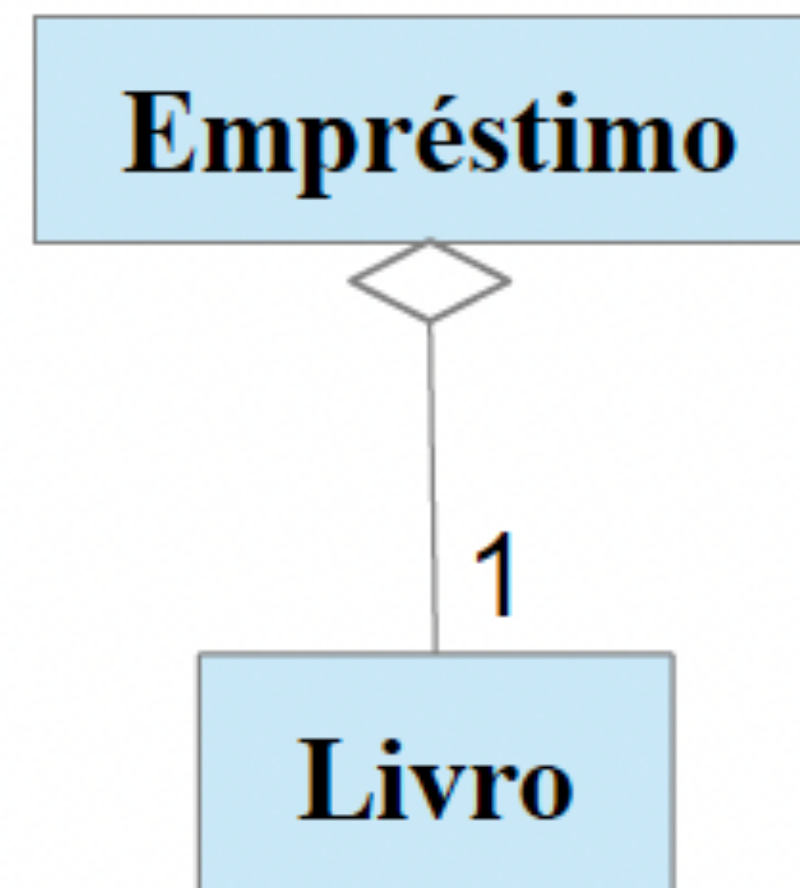


- Uma **Rede** pode ter nenhum ou muitos **Computadores**
- Um computador existe independentemente de uma rede
- Um computador pode estar ligado a mais de uma rede ao mesmo tempo



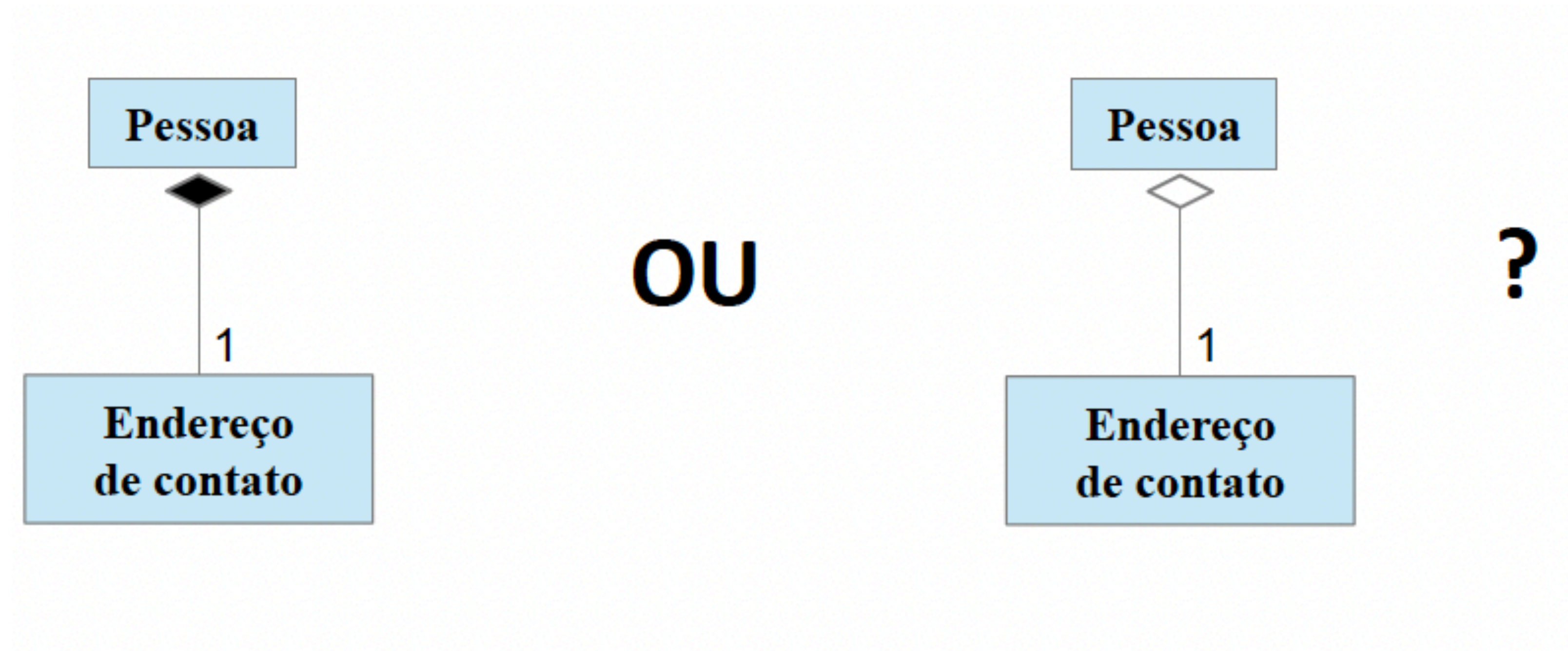


- Um **Livro** existe independente de um **Empréstimo**
- Porém, um **Empréstimo** precisa ter pelo menos um **Livro**



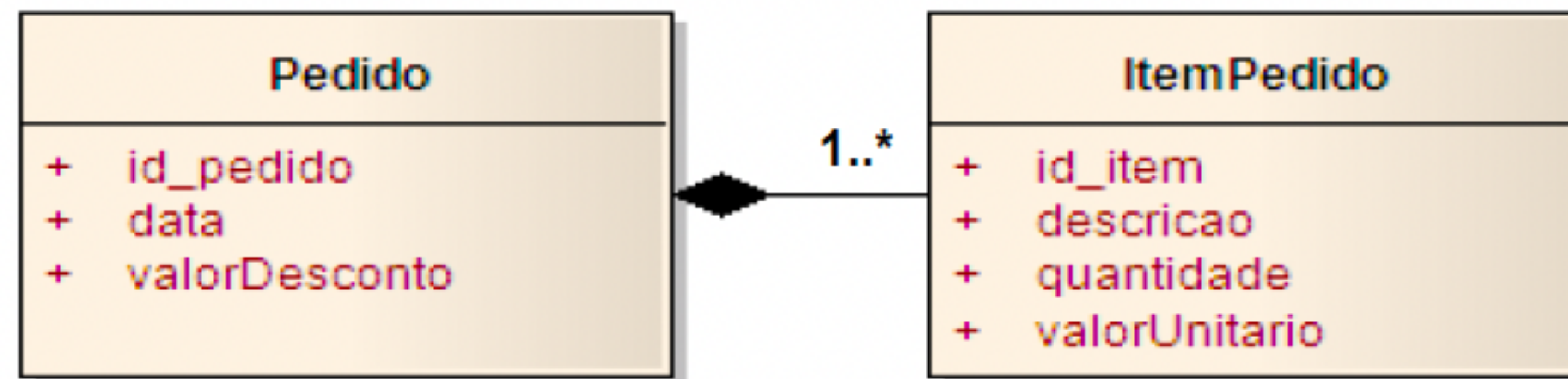


- Quando usar composição ou agregação?
 - Depende dos requisitos do projeto
 - Deve-se interpretar o problema e justificar a escolha

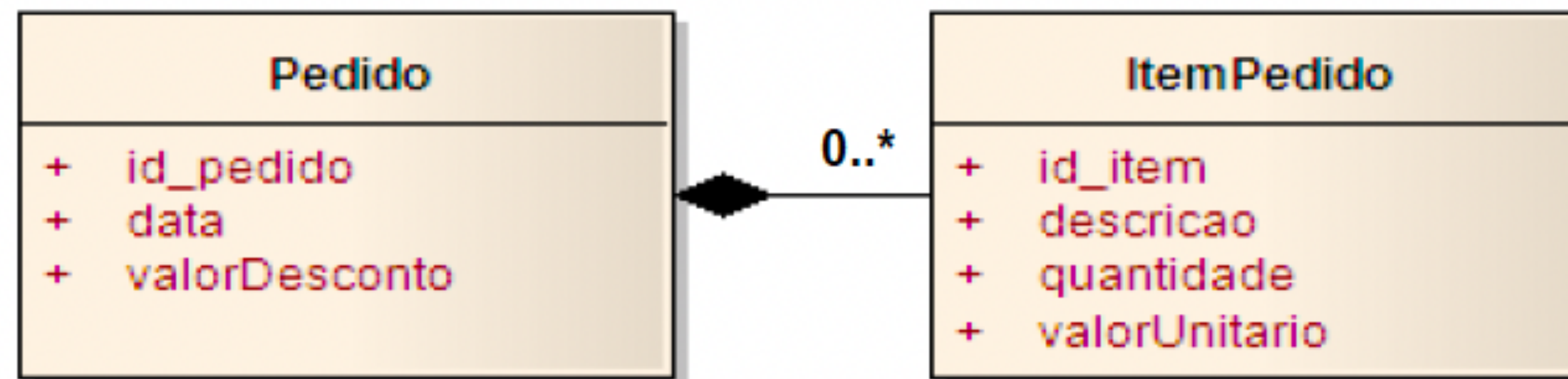




- Um **Pedido** é composto por um ou vários **ItemPedido**
 - Pela cardinalidade, um Pedido precisa ter ao menos um ItemPedido
 - Composição indica que ItemPedido só existe com Pedido



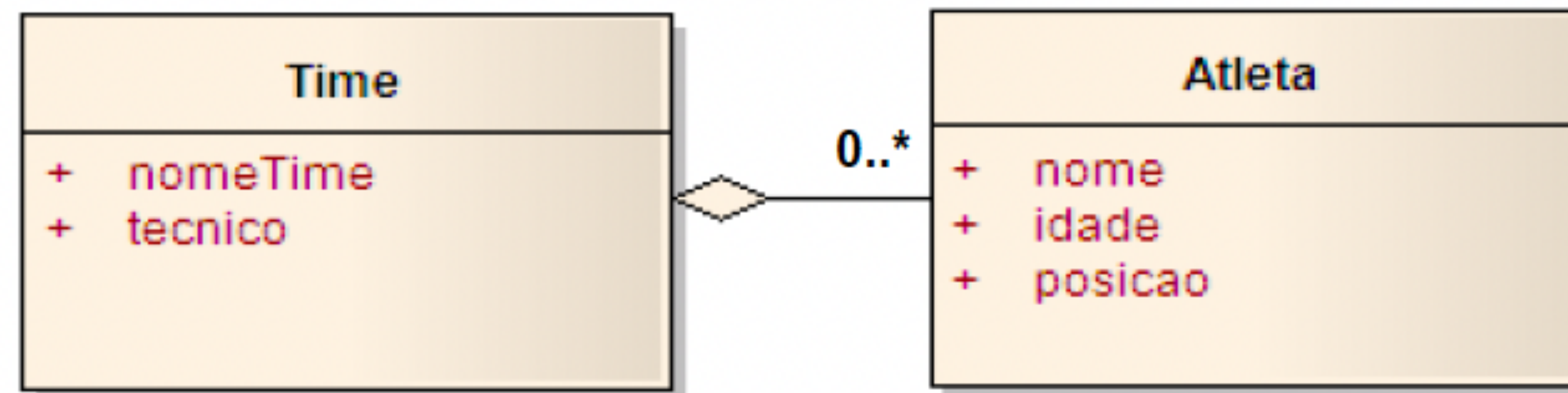
- Um **Pedido** é composto por um ou vários **ItemPedido**
 - **Agora**, um Pedido pode não ter ItemPedido
 - Contudo, a composição continua indicando que ItemPedido só existe com Pedido





● Um **Time** é formado por **Atletas**

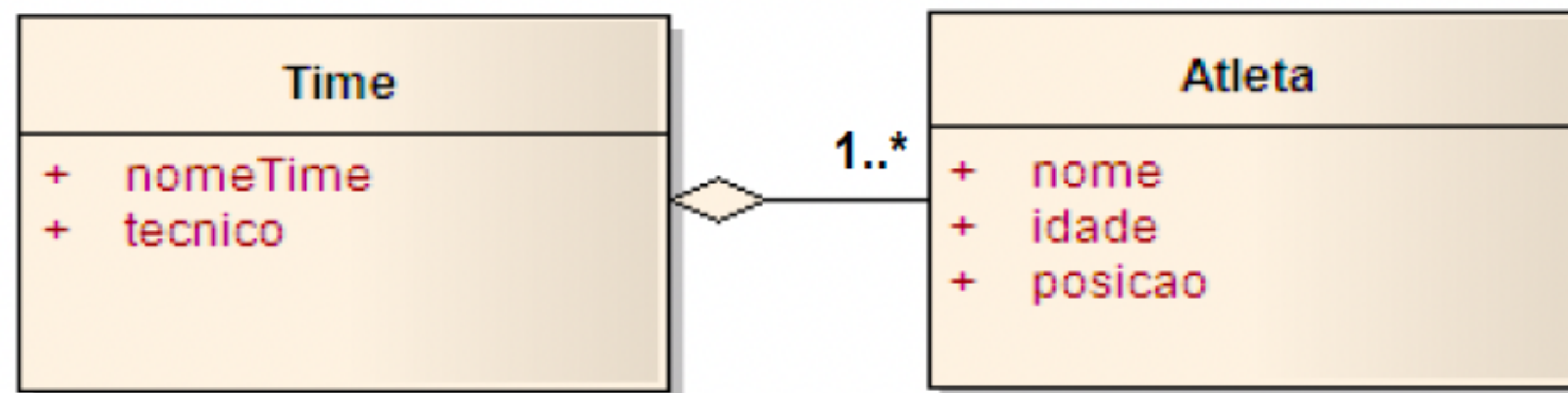
- Pela cardinalidade, um Time pode existir mesmo que não haja Atleta neste time
- Agregação indica que Atleta existe independente da existência de um Time





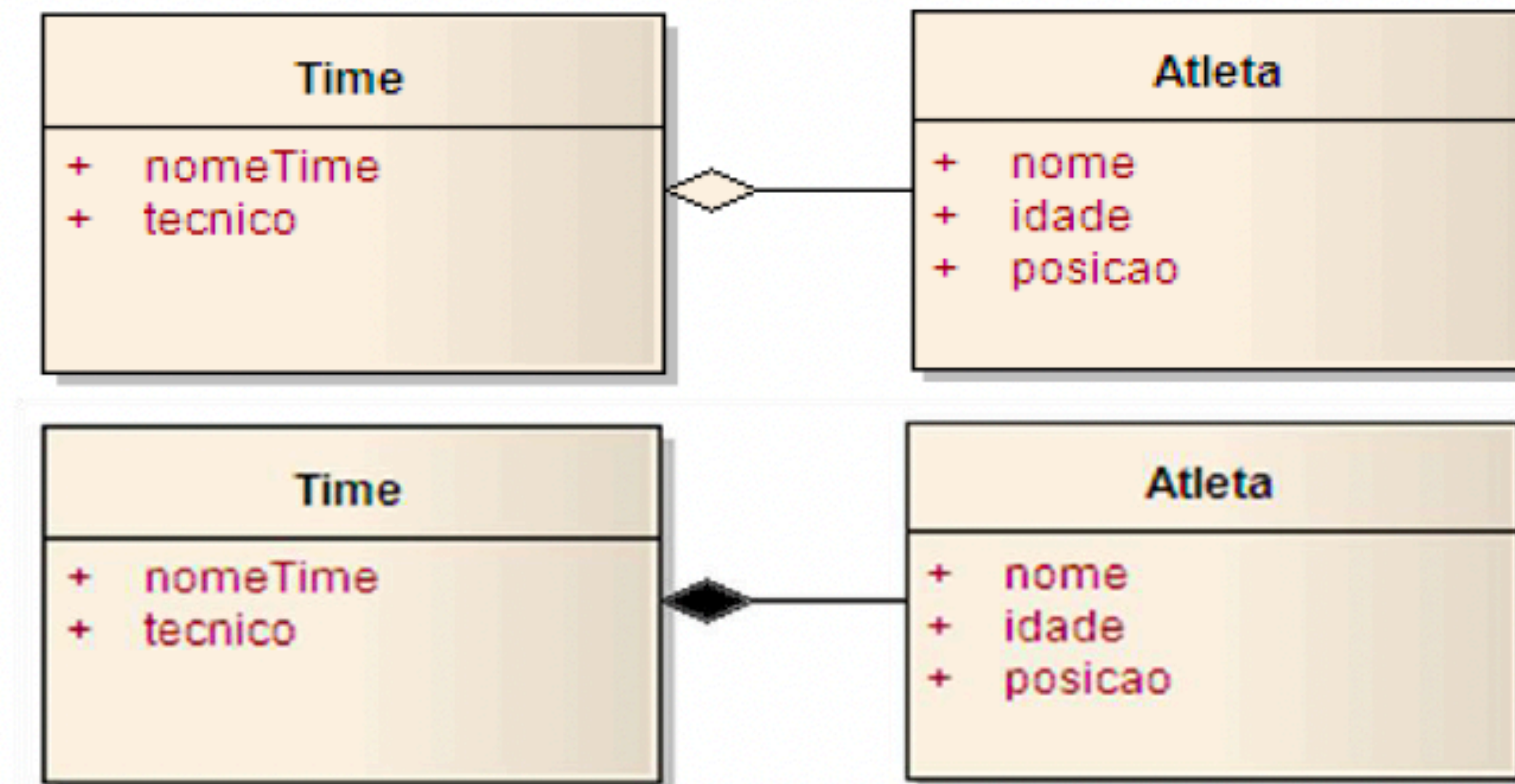
● Um **Time** é formado por **Atletas**

- **Agora**, um Time precisa conter pelo menos um Atleta
- Contudo, a agregação continua indicando que Atleta existe independente da existência de um Time



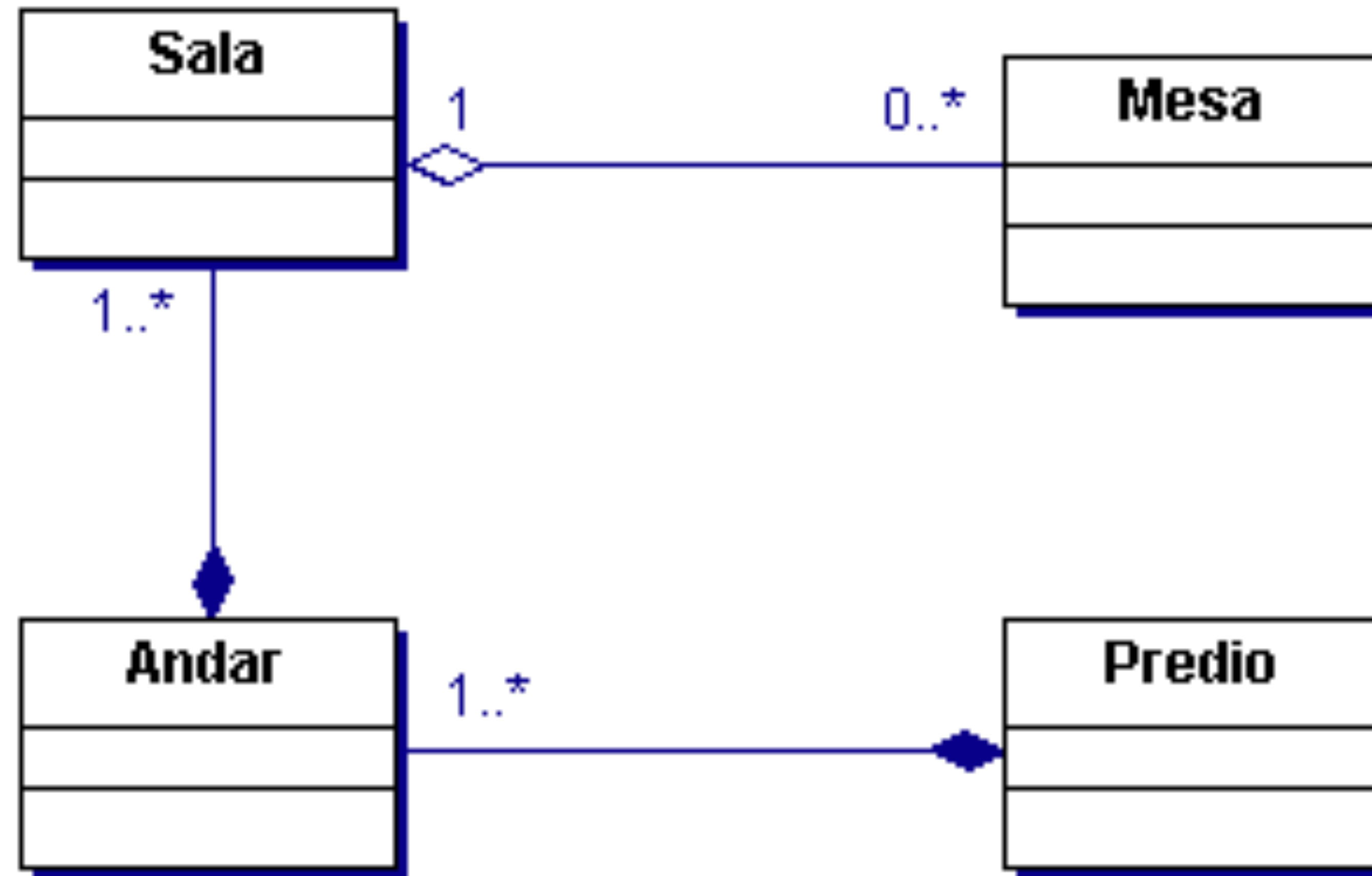


- A semântica (significado) é interpretada pelo projetista
 - Modela o sistema de acordo com sua compreensão e conveniência
 - O mesmo problema pode ser interpretado como composição ou agregação



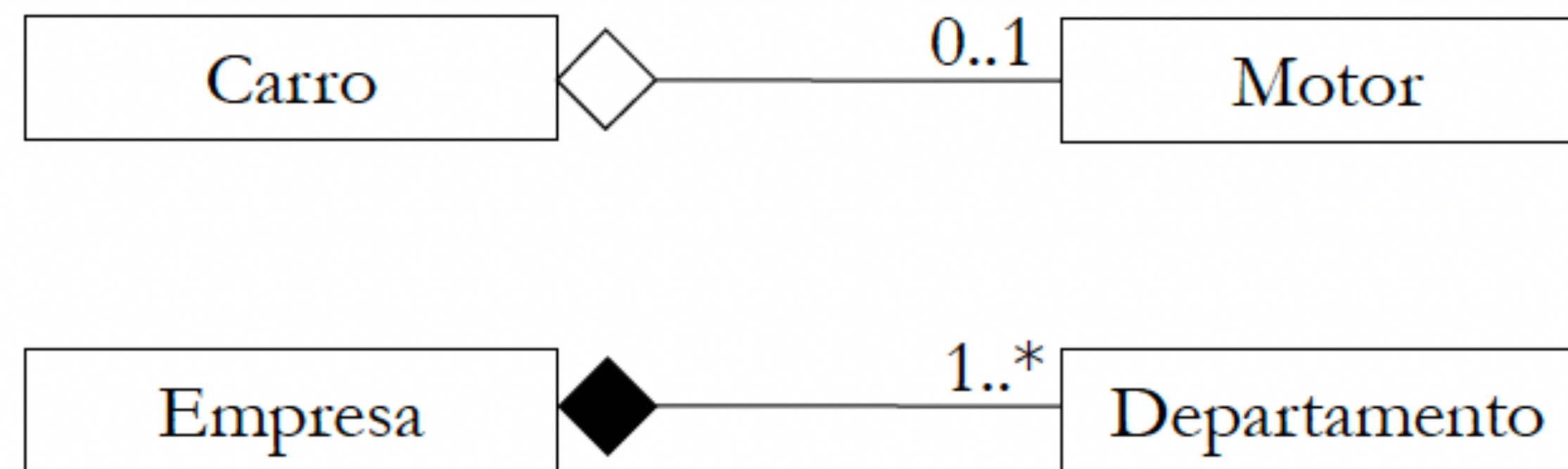


● Outros exemplos





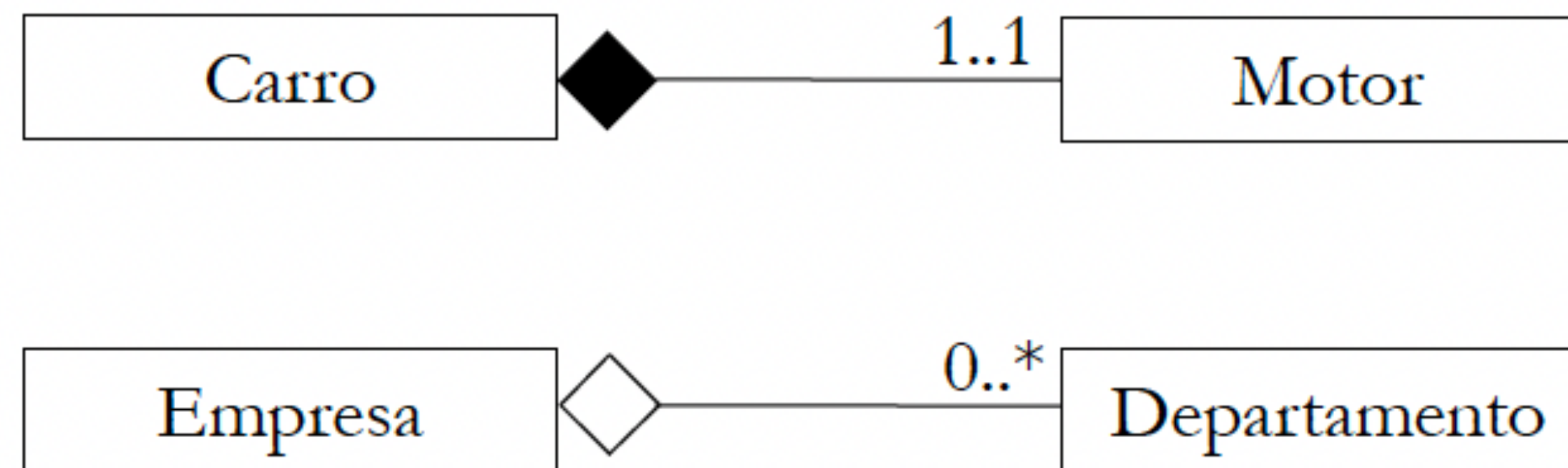
● Outros exemplos





● Outros exemplos

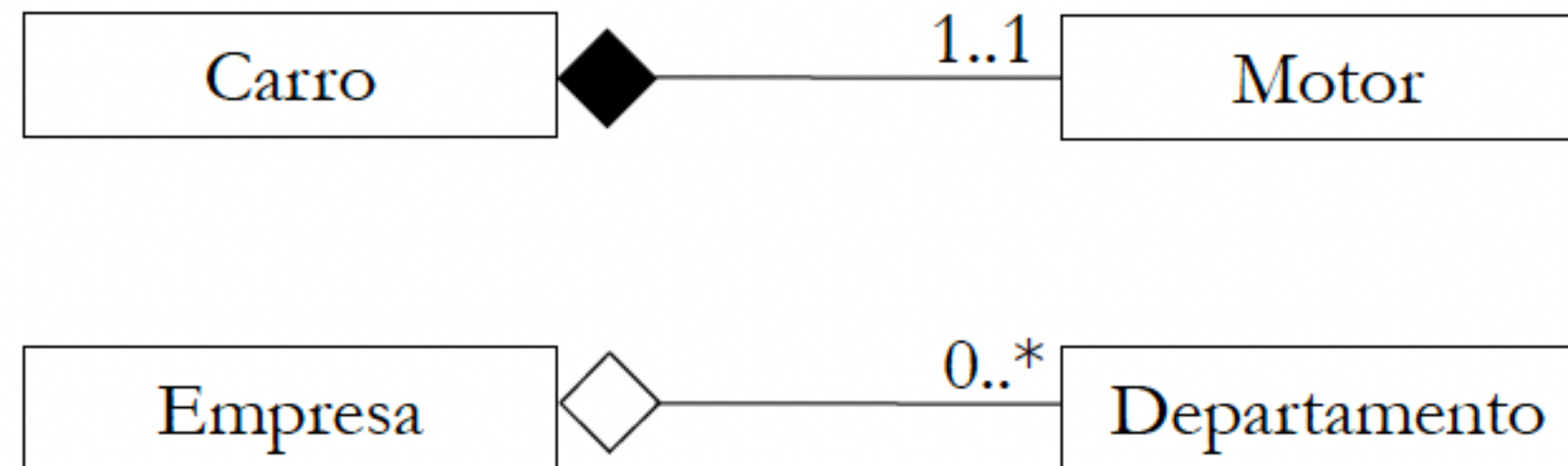
- Se invertermos, fica correto?





● Outros exemplos

- Se invertermos, fica correto?
 - Sim! O projeto é seu!
 - Desde que satisfaça as características do problema

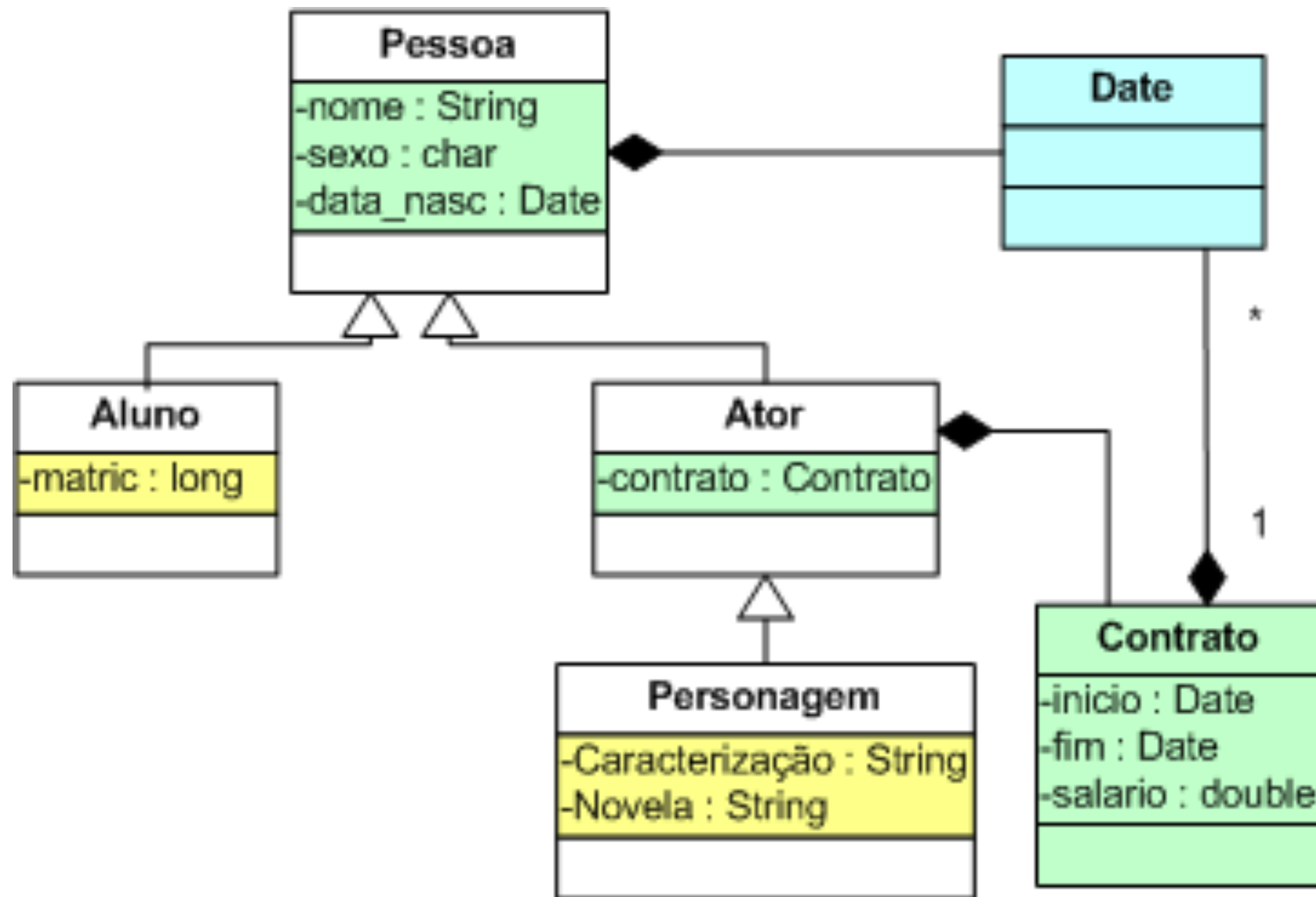




- Como saber que tipo de relacionamento deve ser utilizado?
 - Existem atributos ou métodos em comum entre as classes? Ou seja, uma classe “é do tipo” da outra?
 - **Sim**: Isso é **HERANÇA**
 - **Não**: Existe relação todo-parte?
 - **Não**: Isso é uma ASSOCIAÇÃO SIMPLES
 - **Sim**: A parte vive sem o todo?
 - **Sim**: Isso é uma **AGREGAÇÃO**
 - **Não**: Isso é uma **COMPOSIÇÃO**



● Exemplos





- ◎ Histórico da UML
- ◎ Diagrama de classes
- ◎ Representação de classes
 - Atributos e métodos
 - Tipos de acesso e modificadores
- ◎ Nesta aula foram vistos os principais relacionamentos entre classes
 - Herança, Implementação, Associação, Agregação e Composição



◎ Plugin para o NetBeans

- Ferramentas -> Plugins-> PlantUML
- Instalação do Graphviz

◎ Outros

- RationalRose
- Yed
- StarUML
- Dia
- ArgoUML
- Microsoft Visio
- Enterprise Architect



- DEITEL, H. M. & DEITEL, P.J. "Java : como programar", Bookman, 2017.
- Agradecimentos. Material baseado nos slides:
 - Luiz E. Virgilio da Silva. Notas de Aula de Programação Orientada a Objetos. Classes e Objetos (ICMC/USP).